ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Ecole Reconnue par l'Etat

RAPPORT DE PROJET DE FIN D'ANNÉE (PFA)

GraphRAG Hybride sur une Thèse de Volcanologie :



Réalisé par : Khadija Boumaji et Inas Mourid

Filière : 4ème Année Ingénierie – Data et Intelligence Artificielle

Encadrant pédagogique : Dr. El Mkhalet Mouna

Encadrant entreprenariat: Benyahya zineb

Gestion de projet : Monsif Dounia

Année universitaire : 2025 – 2026

Acknowledgments

We would like to express our deep gratitude to all those who contributed, directly or indirectly, to the completion of this Final Year Project.

Our sincerest thanks go first and foremost to our academic supervisor, Dr. El Mkhalet Mouna, for her availability, wise counsel, scientific rigor, and constant support throughout this project. Her guidance enabled us to structure our work with method and ambition.

We are also grateful to our Project Management instructor for his invaluable teachings on project structuring, risk management, and planning methodologies. His course provided us with the essential tools to approach this project with a professional and structured mindset.

We extend our thanks to our Entrepreneurship instructor for inspiring us to think beyond the technical aspects and to consider the market potential and business viability of our work. His insights into innovation, value proposition, and business modeling were instrumental in shaping the entrepreneurial dimension of this project.

We also thank the EMSI administration for the quality of the training provided and for the resources made available to us, enabling us to successfully carry out our academic projects.

Our appreciation goes to the jury members who kindly devoted their time and expertise to the evaluation of this work.

Finally, we express our gratitude to our families and friends for their unwavering moral support during this intensive period of work.

Résumé

Ce projet de fin d'année porte sur la conception et le développement d'un système RAG hybride (Retrieval-Augmented Generation) appliqué à une thèse de doctorat en volcanologie portant sur la structure interne du volcan Piton des Neiges (île de La Réunion) et son potentiel géothermique.

La thèse source, rédigée par Camille Paquez (Université de La Réunion, 2021), constitue un corpus scientifique dense et structuré de plus de 400 pages. Notre travail consiste à transformer ce document en une base de connaissances interrogeable de manière intelligente, en combinant deux paradigmes complémentaires : la recherche vectorielle sémantique et la recherche par graphe de connaissances.

La méthodologie adoptée comprend cinq étapes principales : (1) le nettoyage et la préparation du texte, (2) le chunking comparatif selon sept méthodes avec sélection de la méthode optimale (M4 - Sliding Window), (3) la génération d'embeddings avec le modèle all-mpnet-base-v2 et évaluation comparative de six méthodes de retrieval (BM25 retenu avec le meilleur score global de 0.660), (4) la construction d'un graphe de connaissances dans Neo4j Aura avec détection de communautés par l'algorithme de Louvain (26 communautés, modularité 0.906), et (5) l'orchestration agentique via Q-Learning pour le routage intelligent des requêtes entre les deux stores.

Le système est exposé via une API FastAPI et un frontend interactif visualisant le graphe RAG, les communautés sémantiques et les chemins de décision de l'agent. La fusion Graph + Vector atteint un score de fusion de 0.87. Le déploiement final est réalisé via Docker.

Mots-clés : RAG, GraphRAG, Neo4j, embeddings, BM25, Louvain, Q-Learning, FastAPI, Piton des Neiges, géothermie, volcanologie.

Abstract

This final year project focuses on the design and development of a hybrid Retrieval-Augmented Generation (RAG) system applied to a doctoral thesis in volcanology on the internal structure of the Piton des Neiges volcano (Réunion Island) and its geothermal potential.

The source thesis, authored by Camille Paquez (University of Réunion, 2021), constitutes a dense and structured scientific corpus of over 400 pages. Our work consists in transforming this document into an intelligently queryable knowledge base by combining two complementary paradigms: semantic vector search and knowledge graph-based search.

The adopted methodology comprises five main stages: (1) text cleaning and preprocessing, (2) comparative chunking across seven methods with selection of the optimal method (M4 - Sliding Window, avg. 1,890 chars, best intra-similarity 0.511), (3) embedding generation using the all-mpnet-base-v2 model and comparative evaluation of six retrieval methods (BM25 retained with the best global score of 0.660), (4) construction of a knowledge graph in Neo4j Aura (1,016 entities, 5,200 relationships) with Louvain community detection (26 communities, modularity 0.906), and (5) agentic orchestration via Q-Learning for intelligent query routing between the two stores.

The system is exposed via a FastAPI backend and an interactive frontend visualizing the RAG graph, semantic communities, and the agent's decision paths. The Graph + Vector fusion achieves a final fusion score of 0.87 (alpha=0.60). Final deployment is achieved through Docker.

Keywords: RAG, GraphRAG, Neo4j, embeddings, BM25, Louvain, Q-Learning, FastAPI, Piton des Neiges, geothermal energy, volcanology.

THESIS SUMMARY – Piton des Neiges Volcano

1. Presentation of the Thesis

Title: Structure interne d'un volcan basaltique (Le Piton des Neiges, Ile de La Réunion), pour l'évaluation de son potentiel géothermique

Author: Camille Paquez

Institution: Université de La Réunion – Laboratoire LGSR / IPGP

Defense Date: September 16, 2021

HAL ID: tel-03934766

The thesis investigates the volcanic architecture of Piton des Neiges through multi-technique geochronology (U-Pb on zircon, (U-Th)/He on apatite), geological mapping at 1/25,000 scale, and conceptual modeling of the hydrothermal system. Key findings include: polyphase plutonism (~2 Ma, ~1.4 Ma, ~0.7 Ma, ~0.15 Ma), dating of the main caldera at 188 ± 5 Ka, and refinement of the last volcanic activity to ~27 Ka. The thesis identifies structural controls on the hydrothermal system, particularly the role of breccia levels, caldera faults, and the N030 rift zone.

2. Context and Motivation (Why This Research Was Needed)

Réunion is a French volcanic island located in the Indian Ocean. This island has an energy dependency rate of 87.5% due to the massive importation of fossil fuels for its electricity production. Faced with this situation, the French government has set an ambitious objective to achieve energy autonomy for non-interconnected zones such as Réunion by 2030. In this context, geothermal energy appears as a particularly interesting renewable energy source because it is available continuously, twenty-four hours a day, three hundred and sixty-five days a year, unlike solar and wind energy which are intermittent. Piton des Neiges, the inactive volcano of the island, presents numerous signs of hydrothermalism at the surface, including thermal springs and degassing phenomena, suggesting the presence of exploitable geothermal potential at depth. Previous drilling campaigns, notably the SGTH-1 and SLZ-1 boreholes, have revealed elevated geothermal gradients, confirming the existence of a significant heat source at relatively shallow depth. However, these boreholes did not encounter sufficient permeability to allow a circulation of hydrothermal fluids suitable for industrial exploitation. The central problem therefore lies in the difficulty of identifying permeable zones within the volcanic massif, whose internal structure is complex and still poorly understood. It thus appears necessary to improve the geological and structural knowledge of Piton des Neiges in order to locate fracture zones likely to present the required permeability to host hydrothermal circulations and enable viable geothermal exploitation. It is precisely with this objective that this thesis work is undertaken, aiming to provide new constraints on the architecture of the volcano and its associated hydrothermal system.

2. Research Questions and Objectives

The main research questions addressed in this thesis revolve around the internal architecture of Piton des Neiges and its implications for geothermal exploration. What is the precise internal structure of this basaltic volcano? Does one or several calderas exist within the massif, and what is their exact extent? What is the nature, depth and age of the heat source sustaining the hydrothermal system observed at the surface? What are the structural and lithological controls on fluid circulation within the edifice? Where are the permeable zones located that could be favorable to geothermal exploitation?

To answer these questions, the thesis pursued four main objectives. The first objective was to produce a new geological map of the three cirques at the 1/25,000 scale, based on extensive field reconnaissance including the most inaccessible areas of the massif. This new cartographic work aimed to identify lithological discontinuities and structural features that could control fluid circulation. The second objective consisted in dating key geological units that had previously resisted dating attempts, including the syenite intrusions, the gabbroic plutonic complex, and the trachytic pyroclastic deposits. The third objective was to identify the faults and limits of the calderas, and to determine whether the cirque boundaries correspond to caldera faults or to other geological structures. Finally, the fourth objective was to propose an updated conceptual model of the hydrothermal system integrating the new geological and geochronological constraints, in order to identify priority zones for future exploration drilling.

3. Methodology

The methodology adopted in this thesis combined three complementary approaches. A geological mapping campaign was conducted over more than two years across the three cirques, covering over 250 km^2 through extensive field reconnaissance and aerial photogrammetry, resulting in a new geological map at the 1/25,000 scale. A multi-technique geochronological study was carried out on previously undated key units using zircon U-Pb, phlogopite 40Ar/39Ar, zircon and apatite (U-Th)/He, and zircon double-dating methods, allowing the reconstruction of the volcano's cooling and eruptive history. A structural analysis was also performed to identify faults, fractures, unconformities and breccia units, integrating all data into a coherent model of the volcano's internal architecture and its hydrothermal system.

4-Overview of the Research Approach and Key Findings

The methodology combined extensive geological mapping across the three cirques, multi-technique geochronology including U-Pb zircon, 40Ar/39Ar phlogopite and (U-Th)/He zircon and apatite dating, and detailed structural analysis of faults, fractures and breccia units. This approach produced several significant results: the identification of polyphase plutonism with magmatic injections at ~2.0 Ma, ~1.4 Ma and ~0.7 Ma, the dating of the main caldera at 188.2 ± 10.4 ka, the refinement of the last volcanic activity to ~27 ka, and a new 1/25,000 geological

map revealing a restricted caldera of ~35 km^2 while demonstrating that the cirques are not calderas but result from erosion, flank collapses and breccia units. The discussion of these findings highlights that the heat source is the cooling magma chamber from the last eruption at ~27 ka, that the hydrothermal system is structurally controlled by caldera faults and the N030 rift zone with lithological control by breccia units acting as reservoirs, and that five priority zones were identified for future exploration. Together, these results provide a significantly improved understanding of the volcano's architecture and its geothermal potential, offering a scientific foundation for targeted exploration strategies on the island of La Réunion.

5-Limitations and Perspectives

The study presents several limitations, including the absence of deep geophysical data to confirm structures at depth, the reliance on expert-curated dictionaries rather than a dedicated NER model for entity extraction, the use of TF-IDF as a proxy for relevance instead of human-annotated ground truth, and the restriction to a single document corpus. To address these challenges, future research should prioritize the acquisition of geophysical surveys such as ERT and AMT to validate the structural model at depth, the development of a fine-tuned multilingual NER model for automatic entity extraction, the extension to multi-document RAG with cross-document reasoning, and the implementation of a full CI/CD pipeline with Docker deployment for reproducibility. Additionally, numerical simulations of fluid flow and heat transfer, thermo-barometric analyses of magma chamber conditions, and sociological studies to improve public acceptance of geothermal projects would provide crucial complementary insights for advancing both the scientific understanding of the volcano and the practical development of geothermal energy on La Réunion Island.

Liste des Figures

Référence	Description	Chapitre
Figure 1	Architecture globale du système GraphRAG hybride	Partie III
Figure 2	Pipeline de nettoyage et préparation du texte	Partie III – Ch. 1
Figure 3	Comparaison des 7 méthodes de chunking (métriques)	Partie III – Ch. 2
Figure 4	Distribution de la longueur des chunks par méthode	Partie III – Ch. 2
Figure 5	Résultats comparatifs des 6 méthodes de retrieval	Partie III – Ch. 3
Figure 6	Graphe de connaissances Neo4j Aura – vue globale (1,074 noeuds, 6,754 relations)	Partie III – Ch. 4
Figure 7	Interface de détection de communautés Louvain – 26 communautés, modularité 0.906	Partie III – Ch. 4
Figure 8	Dashboard frontend – Vue RAG Aura (Neo4j Knowledge Network)	Partie III – Ch. 4
Figure 9	Dashboard frontend – Cluster Members Overview & Semantic Paths	Partie III – Ch. 4
Figure 10	Diagramme de l'agent Q-Learning et processus de routage	Partie III – Ch. 5
Figure 11	Formule de fusion Graph + Vector (score final 0.87)	Partie III – Ch. 5
Figure 12	Architecture FastAPI – Endpoints et flux de données	Partie III – Ch. 6
Figure 13	Capture Swagger UI / Documentation API	Partie III – Ch. 6
Figure 14	Schéma de déploiement Docker (à compléter)	Partie III – Ch. 7
Figure 15	Canvas de proposition de valeur – Solution RAG	Partie I
Figure 16	Matrice SWOT du projet entrepreneurial	Partie I
Figure 17	Analyse PESTEL – Environnement du marché RAG	Partie I
Figure 18	Diagramme de Gantt – Planification du projet	Partie II
Figure 19	Matrice des risques (Probabilité × Impact)	Partie II

Liste des Tableaux

Référence	Description	Chapitre
Tableau 1	Fiche descriptive du projet NutriTrack	Partie II
Tableau 2	Matrice RACI – Répartition des responsabilités	Partie II
Tableau 3	Planning macro – Tâches et jalons	Partie II
Tableau 4	Analyse des risques R1–R10	Partie II
Tableau 5	Comparaison des 7 méthodes de chunking	Partie III – Ch. 2
Tableau 6	Métriques de retrieval – 6 méthodes (P@5, R@5, F1, MRR, nDCG@5)	Partie III – Ch. 3
Tableau 7	Statistiques du graphe Neo4j (types de noeuds et relations)	Partie III – Ch. 4
Tableau 8	Top 10 communautés Louvain par taille	Partie III – Ch. 4
Tableau 9	Q-table initiale de l'agent (états × actions)	Partie III – Ch. 5
Tableau 10	Endpoints API FastAPI – Description et paramètres	Partie III – Ch. 6
Tableau 11	Comparaison des solutions concurrentes (RAG)	Partie I
Tableau 12	Étude de faisabilité financière – Investissement initial	Partie I
Tableau 13	Charges fixes et variables mensuelles	Partie I
Tableau 14	Résultat prévisionnel et seuil de rentabilité	Partie I

Liste des Acronymes

Acronyme	Signification
API	Application Programming Interface
BM25	Best Match 25 – Algorithme de ranking probabiliste
CORS	Cross-Origin Resource Sharing
DPR	Dense Passage Retrieval
FAISS	Facebook AI Similarity Search
FastAPI	Framework Python pour APIs REST rapides
F1	F1-Score (Harmonic mean of Precision and Recall)
GNN	Graph Neural Network
IR	Information Retrieval
LLM	Large Language Model
Louvain	Algorithme de détection de communautés (Louvain et al., 2008)
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
Neo4j	Graph database platform (Aura cloud)
NER	Named Entity Recognition
NLP	Natural Language Processing
PCA	Principal Component Analysis
PFA	Projet de Fin d'Année
PMI	Pointwise Mutual Information
RAG	Retrieval-Augmented Generation
RL	Reinforcement Learning
RACI	Responsible, Accountable, Consulted, Informed
REST	Representational State Transfer
SAAS	Software As A Service
TF-IDF	Term Frequency – Inverse Document Frequency
UI/UX	User Interface / User Experience
U-Pb	Uranium-Lead dating method
(U-Th)/He	Uranium-Thorium/Helium thermochronology
ZDD	Zircon Double Dating

Table des Matières

Remerciements **2**
Résumé **3**
Abstract **4**
Thèse étudiée – Objectifs et Limites **5**
Liste des Figures **7**
Liste des Tableaux **9**
Liste des Acronymes **10**
Table des Matières **11**

PARTIE I – ENTREPRENEURIAT **13**
1. Contexte et Identification du Problème 13
2. Présentation de l'Idée et de la Solution 14
3. Origine et Génération de l'Idée Entrepreneuriale 15
4. Étude de Marché 16
4A. Analyse de la demande 16
4B. Analyse de la concurrence 16
4C. Analyse des fournisseurs 17
4D. Lois et réglementations 17
4E. Outils d'analyse (SWOT, PESTEL, Porter) 17
5. Marketing Stratégique 18
6. Étude de Faisabilité Technique 19
7. Étude de Faisabilité Financière 19
8. Aspect Juridique 20
9. Stratégie de Développement 20

PARTIE II – GESTION DE PROJET (NutriTrack) **22**
1. Cycle de Vie et Phases du Projet 22
2. Fiche Projet Structurée 22
3. Matrice RACI 24
4. Planification – Tâches et Jalons 25
5. Analyse des Risques (Pareto) 26
6. Outils de Pilotage et de Suivi 27

PARTIE III – RAPPORT TECHNIQUE (GraphRAG Volcan) **29**
Chapter 1. Introduction & Background 29
Chapter 2. Data Preprocessing & Cleaning 33
Chapter 3. Chunking Strategy – Comparative Study 37
Chapter 4. Embedding & Retrieval – Comparative Study 43
Chapter 5. Knowledge Graph Construction (Neo4j Aura) 49

Chapter 6. Louvain Community Detection....49
Chapter 7. Q-Learning Agentic RAG....55
Chapter 8. FastAPI Backend Architecture....56
Chapter 9. Frontend Dashboard....56
Chapter 10. Deployment (Docker)....65
Chapter 11. Results & Discussion....67
Chapter 12. Conclusion....69

Bibliographie....70
Webographie....71
Annexes....72

PARTIE I – ENTREPRENEURIAT

Objectif de la Partie Entrepreneuriat

Cette partie a pour objectif de démontrer que le projet GraphRAG constitue non seulement une solution technique performante, mais également une véritable opportunité entrepreneuriale. Elle vise à montrer que ce projet répond à un besoin réel sur le marché, qu'il possède un potentiel commercial crédible, qu'il peut générer de la valeur ajoutée pour ses utilisateurs, qu'il est économiquement viable et qu'il peut être transformé en un projet d'entreprise durable. Cette section explique pourquoi le projet mérite d'exister sur le marché et comment il peut devenir rentable, en s'appuyant sur une analyse structurée du contexte, de l'idée, du marché, de la concurrence, de la faisabilité technique et financière, ainsi que des aspects juridiques et stratégiques nécessaires à sa pérennité.

1. Contexte et Identification du Problème

Nous vivons dans une ère de surcharge informationnelle. Les organisations, institutions académiques et entreprises accumulent des volumes croissants de documents non structurés : thèses, rapports, contrats, manuels techniques, bases de connaissances internes. Selon une étude de FactMR, le marché mondial de la gestion des connaissances (Knowledge Management) était estimé à 773,6 milliards de dollars en 2024, avec une croissance prévue de plus de 16% par an pour la prochaine décennie . Par ailleurs, le marché des outils de gestion des connaissances alimentés par l'IA est évalué à 17,38 milliards de dollars en 2025 et devrait atteindre 31,26 milliards de dollars d'ici 2032, soit un taux de croissance annuel de 8,74% . Ces chiffres illustrent l'ampleur du défi et l'urgence d'innover dans ce domaine.

Secteur concerné :

Le secteur visé est celui des technologies de l'information et de l'intelligence artificielle appliquée à la gestion des connaissances (Knowledge Management). Ce marché se situe à l'intersection du traitement automatique du langage naturel (NLP), du Machine Learning, des bases de données graphes et des APIs d'intelligence artificielle. Ce positionnement stratégique est conforté par l'essor des graphes de connaissances d'entreprise, dont le marché mondial, évalué à 890 millions de dollars en 2025, devrait atteindre 6,55 milliards de dollars d'ici 2036, avec un taux de croissance annuel de 20,1% . Cette croissance est portée par la demande croissante d'infrastructures de données fiables pour les IA et l'accélération de l'adoption des workflows GraphRAG dans les entreprises .

Tendances du marché :

Le marché du Retrieval-Augmented Generation (RAG) connaît une expansion exceptionnelle. Selon Acumen Research and Consulting, le marché mondial du RAG était évalué à 1,99 milliard de dollars en 2025 et devrait atteindre 101,63 milliards de dollars d'ici 2035, avec un taux de croissance annuel composé de 48,2% . D'autres analystes estiment ce marché à 2,3 milliards de dollars en 2025 avec des prévisions atteignant 41,93 milliards de dollars d'ici 2033 . Cette croissance est alimentée par plusieurs facteurs convergents.
L'adoption massive des grands modèles de langage (LLMs) comme ChatGPT, Claude et Gemini s'accompagne d'un besoin urgent de "grounding" sur des données privées et spécifiques, afin de garantir la fiabilité des réponses générées. Le marché de l'IA à lui seul est estimé à 370 milliards

de dollars en 2025 avec une croissance prévue de plus de 30% par an pour les sept prochaines années . Selon une enquête menée par l'APQC auprès de 340 experts en gestion des connaissances, 41% des répondants ont identifié l'intégration de l'IA et des technologies intelligentes comme leur priorité absolue pour 2025 . Par ailleurs, l'essor des agents IA et des workflows GraphRAG est soutenu par l'émergence de plateformes cloud-native qui réduisent les coûts d'exploitation, avec des projets open-source comme graphiti dépassant les 24 000 étoiles sur GitHub, témoignant de l'intérêt massif de la communauté technique .

Identification du problème :

Malgré ces avancées, un problème fondamental persiste : les LLMs génériques, bien que performants, restent sujets à des hallucinations. Une étude récente menée sur 14 modèles de langage et plus de 150 000 générations a révélé que même les modèles les plus performants produisent des hallucinations factuelles allant parfois jusqu'à 86% des affirmations générées selon le domaine d'application . Dans le domaine médical, une autre étude a montré que 58,5% des citations générées par ChatGPT étaient entièrement fabriquées . Dans un contexte professionnel ou académique, où l'exactitude des informations est primordiale, ces erreurs sont rédhibitoires.

Parallèlement, les systèmes RAG purement vectoriels peinent à capturer les relations structurelles complexes entre les concepts. Ils excellent dans la recherche sémantique mais échouent à restituer des liens logiques ou hiérarchiques, ce qui est indispensable dans des corpus spécialisés comme les thèses scientifiques, les rapports techniques ou les dossiers juridiques. Il manque aujourd'hui des solutions robustes combinant efficacement l'approche vectorielle et l'approche graphe dans un pipeline complet, accessible, documenté et adaptable.

Limites des solutions existantes :

Face à ce constat, les solutions actuelles présentent des lacunes significatives. LlamaIndex et LangChain, bien que très populaires, restent complexes à configurer et souffrent d'abstractions lourdes qui ralentissent leur adoption par les équipes techniques non spécialisées. La solution officielle Neo4j GraphRAG, bien que puissante, impose des licences payantes pour accéder à ses fonctionnalités avancées, ce qui la rend inaccessible aux petites structures, aux chercheurs indépendants ou aux établissements académiques disposant de budgets limités. Par ailleurs, les outils propriétaires comme Azure AI Search ou Pinecone, s'ils offrent des performances élevées, présentent des coûts récurrents importants qui augmentent avec le volume de données traitées. L'absence de solutions hybrides open-source intégrant un agent de routage adaptatif, capable de décider dynamiquement entre la recherche vectorielle et la recherche par graphe selon la nature de la requête, constitue une lacune majeure du marché actuel . Les entreprises et les institutions de recherche se trouvent donc dans une impasse : les frameworks libres sont trop complexes, les solutions propriétaires trop coûteuses, et aucune ne propose une approche véritablement intégrée.

2. Présentation de l'Idée et de la Solution

Notre solution est un système RAG hybride Graph + Vector open-source, agentique, applicable à tout corpus scientifique structuré. Le projet pilote utilise la thèse du Piton des Neiges comme corpus de démonstration.

Description de la solution :

Le système permet d'interroger un corpus scientifique via une interface naturelle. L'agent Q-Learning analyse chaque requête, décide de la stratégie optimale (vector / graph / hybride) et retourne une réponse contextualisée et exacte .

Fonctionnalités principales :

- Ingestion et nettoyage automatique de documents texte
- Chunking adaptatif avec 7 stratégies comparées
- Recherche sémantique vectorielle (FAISS + BM25)
- Graphe de connaissances interactif (Neo4j Aura)
- Détection de communautés sémantiques (Louvain)
- Agent de routage adaptatif (Q-Learning)
- API REST FastAPI
- Frontend interactif (HTML/JS/ D3.js)

Technologies utilisées :

- Python 3.13, FastAPI, Uvicorn
- Sentence-Transformers (all-mpnet-base-v2, 768 dims)
- FAISS
- Neo4j Aura (cloud graph database)
- Groq API (Llama-3.3-70b) pour reformulation LLM
- scikit-learn, rank-bm25, networkx, community (python-louvain)
- Docker (déploiement)

Valeur ajoutée et avantage concurrentiel :

- Solution end-to-end en code Python pur, sans dépendance à des frameworks lourds
- Agent Q-Learning adaptatif unique : améliore ses décisions avec le feedback utilisateur
- Visualisation richissime du graphe et des communautés sémantiques
- Score de fusion 0.87 : supérieur aux approches mono-store
- Facilement adaptable à tout domaine (juridique, médical, ingénierie)

4. Origine et Génération de l'Idée Entrepreneuriale

L'idée de ce projet est née d'une observation simple : les chercheurs, ingénieurs et professionnels perdent un temps considérable à retrouver des informations dans leurs propres documents. La consultation d'une thèse de 400 pages pour trouver une donnée précise peut prendre des heures. Ce problème universel constitue une perte de productivité massive pour les organisations.

Cette observation s'inscrit dans un contexte de transformation profonde du marché du travail. Selon McKinsey, 52% des tâches professionnelles pourraient être automatisées par l'IA d'ici 2030. Oracle a récemment licencié près de 21 000 employés, soit 13% de ses effectifs, explicitement en raison de l'adoption de l'IA. Plus largement, plus de 60 000 licenciements liés à l'IA ont été annoncés aux États-Unis en 2024, et 37% des dirigeants déclarent que l'IA a déjà remplacé des employés en 2025.

Derrière ces licenciements se cache une tendance majeure : les entreprises ne se contentent plus d'automatiser des tâches, elles réorganisent leurs modèles d'affaires autour de l'IA. Les grandes

entreprises technologiques investissent massivement dans l'infrastructure IA, mais les organisations de taille moyenne, les institutions académiques et les chercheurs restent dépourvus de solutions accessibles et abordables pour exploiter l'IA sur leurs données privées.

Notre solution comble ce vide en proposant un système RAG hybride open-source, agentique, et adaptable à tout corpus scientifique. L'agent Q-Learning permet un routage intelligent des requêtes entre le store vectoriel et le graphe de connaissances, apprenant à chaque interaction pour améliorer la pertinence des réponses. Cette approche démocratise les capacités des grands acteurs, les rendant accessibles sans dépendance à un fournisseur unique.

4. Étude de Marché – Étude de Faisabilité Commerciale

4 A. Analyse de la Demande

Besoins des utilisateurs :

- **Chercheurs et doctorants** : interrogation rapide de corpus académiques, extraction d'informations clés et réduction du temps de veille documentaire.
- **Ingénieurs et consultants** : accès structuré à la documentation technique et recherche contextuelle sans lecture exhaustive.
- **Entreprises tech et ESN** : intégration RAG dans leurs produits SaaS et réduction des coûts de support technique.
- **Institutions académiques** : valorisation de leurs ressources documentaires et amélioration de l'accessibilité des connaissances.
- **Secteurs spécialisés (géologie, médecine, droit, finance)** : recherche sur corpus techniques et aide à la décision basée sur des données sourcées.

Comportement des consommateurs :

Les utilisateurs cibles sont familiers avec ChatGPT et cherchent des solutions similaires sur leurs données privées. Ils sont prêts à payer pour la confidentialité des données et la précision des réponses (pas d'hallucination).

4B. Analyse de la Concurrence

Concurrent	Type de solution	Taille	Zone	Ancienneté	Prix	Forces	Faiblesses
LlamaIndex	Framework RAG Python	Startup	USA/ Global	2022	Open source + Enterprise	Flexibilité, écosystème riche, adoption massive	Courbe d'apprentissage élevée, complexité pour les non-experts
LangChain	Framework LLM/RAG	Startup ($>25M)	USA/Global	2022	Open source + LangSmith	Large communauté	Abstractions lourdes, lenteur

Neo4j GraphRAG	Graph RAG officiel	Grande entreprise	USA/Global	2024	Freemium + Enterprise	Intégration Neo4j native performance	Licences payantes
Pinecone	Vector DB SaaS	Startup unicorn	USA/Global	2019	Abonnement $70+/mois	Performance, scalabilité	Coût, vendor lock-in
Azure AI Search	Search + RAG Microsoft	Grande entreprise	Global	2014	Pay-per-use	Intégration Azure	Complexité, coût

4C. Analyse des Fournisseurs

Fournisseurs nécessaires au fonctionnement :

Fournisseur	Service	Prix	Stabilité	Alternative
Neo4j Aura	Graph database cloud	Gratuit (free tier) / $65+/mois	Très haute	Amazon Neptune, ArangoDB
Groq API	LLM (Llama-3.3-70b)	Gratuit (500K tokens/jour)	Haute	Anthropic, Mistral, Ollama local
Hugging Face	Modèles d'embedding	Gratuit (open source)	Très haute	OpenAI Ada, Cohere
FAISS (Meta)	Index vectoriel	Gratuit open source	Très haute	Chroma, Qdrant, Weaviate
Cloud hosting	Déploiement serveur	$10-50/mois (VPS)	Haute	AWS, GCP, Azure, OVH

Le projet ne dépend fortement d'aucun fournisseur unique. Chaque composant a des alternatives viables, ce qui garantit la résilience du système.

4D. Lois et Réglementations

- RGPD (Europe) / Loi 09-08 (Maroc) : protection des données personnelles – applicable si données utilisateurs traitées
- Propriété intellectuelle : vérifier les droits d'utilisation des corpus ingérés
- Réglementation IA (AI Act Europe 2024) : classification à risque minimal pour un système RAG académique
- Licences open-source : FAISS (MIT), sentence-transformers (Apache 2.0), Neo4j Community (GPL)

Les risques juridiques sont faibles pour un usage académique. Pour une commercialisation, il faudra vérifier les droits des corpus ingérés.

4E. Outils d'Analyse du Marché

Matrice SWOT :

	Forces (Strengths)	Faiblesses (Weaknesses)
Internes	Solution hybride unique / Agent adaptatif / Code pur sans dépendances lourdes / Score fusion 0.87	Pas de NER automatique / Scalabilité limitée par Neo4j free tier
	Forces (Opportunities)	Menaces (Threats)

Externes	Marché RAG en forte croissance / LLMs omniprésents / Besoin academia fort	Concurrents bien financés / Évolution rapide des LLMs / Coût infrastructure

Analyse PESTEL :

Dimension	Analyse
Politique	Soutien gouvernemental à l'IA au Maroc (stratégie numérique 2030). Régulation IA en cours (AI Act EU).
Économique	Croissance du marché NLP/RAG estimée à 35% CAGR. Modèles open-source réduisent le coût d'entrée.
Social	Digitalisation des institutions académiques. Demande croissante de solutions IA explicables.
Technologique	Avancées rapides en LLMs multilingues. Maturité des bases vectorielles et graphes.
Écologique	Empreinte carbone des LLMs : réduction via modèles quantifiés et hébergement local.
Légal	RGPD, loi 09-08. Licences open-source compatibles usage commercial avec attribution.

5 Forces de Porter :

Force	Niveau	Analyse
Menace nouveaux entrants	Élevé	Faibles barrières techniques – n'importe quel dev peut créer un RAG basique
Pouvoir fournisseurs	Faible	LLMs et outils open-source abondants
Pouvoir clients	Moyen	Clients académiques fidèles mais sensibles aux prix
Produits substituts	Moyen	ChatGPT avec plugins, Perplexity AI
Rivalité interne	Élevé	LlamaIndex, LangChain, Neo4j GraphRAG – acteurs établis

5. Marketing Stratégique

Segmentation :

Segment	Profil	Âge	Niveau tech	Problème
S1 – Académique	Chercheurs, doctorants, étudiants M2	22-45 ans	Moyen à élevé	Interroger des corpus de recherche
S2 – Entreprises tech	Startups IA, ESN, éditeurs SaaS	25-50 ans	Élevé	Intégrer RAG dans leurs produits
S3 – Industrie spécialisée	Géologie, médecine, droit, finance	30-55 ans	Variable	Accéder rapidement à documentation

Ciblage attendu :

Segment principal cible : S1 (Académique) – segment le plus accessible pour un lancement, le plus homogène, et présentant une croissance forte avec la montée des LLMs dans la recherche.

Positionnement :

Positionnement innovant et accessible : solution RAG hybride open-source, plus précise que les LLMs génériques, plus accessible que les frameworks lourds, avec une interface visuelle unique pour explorer la connaissance.

Mix Marketing (4P) :

- Produit : SaaS RAG + Graph Knowledge Explorer, avec API et dashboard
- Prix : Freemium (usage académique gratuit) + abonnement Enterprise (€99-499/mois)
- Place : Déploiement cloud (VPS), téléchargeable sur GitHub
- Promotion : Publications académiques, GitHub stars, démos YouTube, réseaux académiques

Stratégie de lancement :

- Publier le code en open-source sur GitHub avec documentation complète
- Démo publique sur corpus volcanologique (cas d'usage visible)
- Partenariats avec universités (EMSI, Université Mohammed V, etc.)
- Présence sur Hugging Face Spaces pour démonstration accessible

6. Étude de Faisabilité Technique

Matériel nécessaire :

Élément	Quantité	Usage
Serveur VPS (8 vCPU, 16GB RAM)	1	Hébergement backend + Neo4j
GPU (optionnel, T4/A10)	1	Accélération embeddings
Stockage SSD (100GB)	1	Vector store + graph data
Poste de développement	2	Développement et tests

Profils humains nécessaires :

Profil	Quantité	Mission
Ingénieur ML/NLP	1-2	Pipeline chunking, embedding, retrieval
Ingénieur Backend	1	FastAPI, déploiement Docker
Ingénieur Graph/Data	1	Neo4j, Cypher, Louvain
Développeur Frontend	1	Dashboard Vis.js, HTML/CSS/JS

7. Étude de Faisabilité Financière

1 – Investissement Initial :

Élément d'investissement	Quantité	Coût unitaire (MAD)	Coût total (MAD)
Développement logiciel (3 mois)	3	20,000	60,000
Hébergement / Cloud (6 mois)	6	1,500	9,000
Licences / API (Neo4j Pro)	1	5,000	5,000
Design UI/UX	1	8,000	8,000
Marketing de lancement	1	5,000	5,000
Création juridique	1	3,000	3,000
Autres (tests, outils)	1	5,000	5,000
TOTAL			95,000

2 – Prévisions de Ventes (mensuel) :

Produit / Service	Prix de vente (MAD)	Quantité mensuelle prévue	CA mensuel (MAD)
Abonnement Académique (étudiant)	200	50	10,000
Abonnement Pro (entreprise)	1,500	10	15,000
Intégration API (customisation)	5,000	2	10,000
TOTAL CA MENSUEL			35,000

3 – Charges Variables :

Charge variable	Coût unitaire	Quantité estimée	Coût total (MAD)
API AI / Cloud usage	0.50/req	2,000 req/mois	1,000
Commission paiement	2%	sur CA	700
Support client	forfait	partiel	1,000
Maintenance technique	forfait	mensuel	2,000
TOTAL CHARGES VARIABLES			4,700

4 – Charges Fixes Mensuelles :

Charge fixe	Montant mensuel (MAD)
Salaires (2 devs)	20,000
Hébergement serveur	1,500
Internet / télécom	500
Abonnements logiciels	1,000
Marketing fixe	2,000
Comptabilité	800
TOTAL CHARGES FIXES	25,800

5 – Résultat Prévisionnel :

Élément	Montant (MAD)
Chiffre d'affaires mensuel	35,000
Total charges variables	4,700
Total charges fixes	25,800
Coût total	30,500
RÉSULTAT PRÉVISIONNEL	+4,500

6 – Seuil de Rentabilité :

Élément	Résultat
Charges fixes	25,800 MAD/mois
Taux de marge sur coût variable	(35,000-4,700)/35,000 = 86.6%
Seuil de rentabilité	25,800 / 0.866 = ~29,793 MAD/mois

Analyse : Le projet atteint son seuil de rentabilité dès le 3ème mois avec ~50 abonnés académiques et 10 abonnés Pro. Le BFR est faible car le modèle est SaaS (paiement avant livraison).

8. Aspect Juridique

Forme juridique choisie :

SARL (Société à Responsabilité Limitée) – capital minimum 1 MAD. Ce choix est justifié par : la simplicité de création, la limitation de responsabilité des associés, la flexibilité de gestion pour une équipe de 2-4 personnes, et la crédibilité vis-à-vis des clients entreprises.

Propriété intellectuelle :

- Dépôt de marque auprès de l'OMPIC (Office Marocain de la Propriété Industrielle et Commerciale)
- Code source sous licence MIT pour les composants open-source
- Composants propriétaires (agent Q-Learning, dashboard) protégés par copyright

9. Stratégie de Développement

Possibilités d'évolution et perspectives futures :

- Phase 1 (0-6 mois) : Stabilisation technique, open-source sur GitHub, 100 premiers utilisateurs
- Phase 2 (6-18 mois) : Lancement SaaS, partenariats universités, internationalisation (France, Tunisie)
- Phase 3 (18-36 mois) : Extension à d'autres domaines (médical, juridique), levée de fonds
- Vision long terme : Plateforme de knowledge management IA multidomaine, reconnue internationalement

PARTIE II – GESTION DE PROJET

Cette partie traite du cas d'application proposé par le Pr. Monsif D (EMSI – 4IIR) : le lancement de la plateforme NutriTrack. Elle suit strictement les rubriques définies dans le sujet.

1. Choix du Modèle de Cycle de Vie

Modèle choisi : Agile – Scrum

Justification du choix en fonction du contexte :

Le projet GraphRAG présente plusieurs caractéristiques qui rendent inadapté un cycle en cascade (Waterfall) ou un modèle en V :

- Les besoins techniques ne sont pas totalement clarifiés au départ (choix des méthodes de chunking, embedding, retrieval à valider expérimentalement)
- Les équipes de développement et de recherche ont des visions différentes sur les priorités
- Certaines dépendances techniques (Neo4j Aura, Groq API) ne sont pas encore totalement maîtrisées
- Le risque de retard est élevé en raison de la complexité technique et des dépendances externes
- Le délai est strict (4 mois) → Agile permet de livrer des incréments fonctionnels à chaque sprint

Phases du projet (4 Sprints de 2-3 semaines) :

Phase	Durée	Contenu	Livrable
Sprint 0 – Cadrage	Sem 1-2	Analyse des besoins, spécifications techniques, architecture, choix des technologies	Backlog produit, Architecture technique
Sprint 1 – Core	Sem 3-5	Pipeline de nettoyage, chunking comparatif, embeddings, FAISS	Vector store fonctionnel
Sprint 2 – Graph	Sem 6-8	Conception UX/UI, maquettes, frontend, module suivi	Graphe de connaissances
Sprint 3 – Agent	Sem 9-12	Intégration logistique, IA recommandations, tests	API fonctionnelle
Sprint 4 – Frontend	Sem 12-14	Tests finaux, corrections, marketing, déploiement	Application complète

2. Fiche Projet Structurée

Contexte du projet	Projet de fin d'année en 4ème année Ingénierie IA à l'EMSI Casablanca. Conception d'un système RAG hybride (Graph + Vector) appliqué à une hese de heseation (Piton des Neiges).
Objectif général	Construire et heseat un système RAG agentique fonctionnel en 3 mois, capable de répondre à des questions scientifiques sur le corpus de la hese via une recherche vectorielle, un retrieval par graphe et un routage Q-Learning.
Objectifs spécifiques	1. Nettoyer et préparer le texte de la thèse (normalisation UTF-8, suppression caractères spéciaux, conservation des paragraphes) 2. Comparer 7 méthodes de chunking (Fixed-Size, Sentence-Based, Paragraph-Based, Sliding Window, Semantic, Recursive Split, Token-Based) et sélectionner la meilleure selon les métriques de cohérence intra-chunk et inter-chunk 3. Tester et comparer plusieurs modèles d'embeddings (all-mpnet-base-v2, paraphrase-multilingual-MiniLM-L12-v2, multilingual-e5-large, BAAI/bge-m3, text-embedding-ada-002, Cohere embed-multilingual-v3) et sélectionner le plus performant sur le corpus français spécialisé 4. Générer les embeddings avec le modèle sélectionné et construire l'index FAISS 5. Évaluer 6 méthodes de retrieval (Cosine, FAISS, BM25, DPR, Hybrid Vector+BM25, Reranking) avec les métriques P@5, R@5, F1, MRR, nDCG@5 et sélectionner la meilleure 6. Construire un graphe de connaissances dans Neo4j Aura à partir des entités et relations extraites du corpus 7. Appliquer l'algorithme de Louvain pour la détection de communautés sémantiques (modularité cible > 0.85) 8. Implémenter un agent Q-Learning pour le routage adaptatif entre le store vectoriel et le graphe de connaissances 9. Développer une API FastAPI exposant les endpoints REST et un frontend interactif (HTML/JS/D3.js) 10. Documenter l'ensemble du pipeline et déployer via Docker
Périmètre du projet (inclus)	Pipeline de nettoyage / Chunking heseation / Embeddings FAISS / BM25 / Graphe Neo4j / Louvain / Q-Learning / FastAPI / Frontend HTML/JS/D3.js / Documentation / Docker
Périmètre du projet (exclus)	Intégration LLM avancée (Groq optionnel) / Modèle NER automatique / RAG multi-documents / Déploiement en production cloud
Parties prenantes	Étudiantes (Khadija Bounaji & Inas Mourid) / Encadrante pédagogique (Dr. El Mkhalet Mouna) / Encadrant entreprise
Besoins fonctionnels	Interrogation naturelle du corpus / Recherche sémantique (vectorielle) / Recherche par graphe (Neo4j) / Routage agentique Q-Learning / Visualisation du graphe / Affichage des communautés Louvain
Besoins non fonctionnels	Performance : temps de réponse < 3s / Modularité : architecture extensible / Documentation : heseat / Open-source / Reproductible via Docker

Contraintes (délai, budget, techniques)	Délai : 3 mois / Budget : limité / Technique : dépendances hesea (Neo4j Aura, Groq API) / Environnement : développement local
Livrables attendus	Code source complet / Vector store / Graphe Neo4j / API FastAPI / Frontend interactif / Rapports / Documentation / Déploiement Docker
Planning macro (phases)	Sprint 0 (Sem 1-2) : Cadrage / Sprint 1 (Sem 3-5) : Core / Sprint 2 (Sem 6-8) : Graph / Sprint 3 (Sem 9-12) : Agent / Sprint 4 (Sem 12-14) : Frontend
Critères de réussite	BM25 score > 0.65 / Modularité Louvain > 0.85 / Score fusion Graph+Vector > 0.80 / API fonctionnelle / Frontend interactif / Documentation complète
Risques principaux identifiés	R1: API Neo4j Aura instable / R2: Groq API indisponible / R3: Complexité Q-Learning / R4: Conflit équipes (tech vs recherche) / R5: Bugs heseation
Hypothèses du projet	Neo4j Aura free tier sera disponible / Groq API sera accessible / Les données de la hese seront exploitables / Les méthodes de chunking comparatives donneront des résultats exploitables

3. Matrice RACI

R = Responsible (exécutant) | A = Accountable (garant) | C = Consulted | I = Informed

Activités / Livrables	Chef projet(KHADIJA BOUNAJI ET INAS MOURID)	Dev Backend	Dev Frontend	UX/UI Designer	Marketing	Encadrante	Client (Jury)
Cadrage & Spécifications	A	R	C	C	C	C	C
Nettoyage du texte	A	R	I	I	I	C	I
Chunking comparatif	A	R	C	I	I	C	I
Embeddings comparatif & FAISS	A	R	I	I	I	C	I
BM25 & retrieval	A	R	C	I	I	C	I
Construction graphe Neo4j	A	R	I	I	I	C	I
Louvain communautés	A	R	I	I	I	C	I
Q-Learning agent	A	R	I	I	I	C	I
FastAPI backend	A	R	C	I	I	C	I
Frontend interactif	C	C	R	C	I	I	I
Tests & validation	A	R	R	C	I	C	I
Documentation	A	R	R	I	I	C	I
Déploiement Docker	A	R	C	I	I	C	I
Soutenance	R	R	I	I	I	C	A

Justification des choix critiques

Activité	Justification
Cadrage & Spécifications	Le chef de projet est Accountable car il définit l'architecture et les choix techniques. Le Dev Backend est Responsible car c'est lui qui implémente la solution.
Chunking & Retrieval	Le chef de projet est Accountable de la performance. Le Dev Backend est Responsible car c'est lui qui code et teste les méthodes.
Frontend	Le chef de projet est Accountable de la livraison. Le Dev Frontend est Responsible car c'est lui qui développe l'interface.
Soutenance	Les deux étudiantes sont Responsible car elles présentent le travail. Le Client/Jury est Accountable car c'est lui qui valide le projet.

4. Planification Réaliste – Tâches et Jalons

Code	Tâche	Description	Antériorité	Durée (jours)
A	Analyse des besoins	Recueil et clarification des besoins	–	4
B	Rédaction cahier des charges	Formalisation des exigences	A	3
C	Conception UX/UI	Maquettes et expérience utilisateur	B	5
D	Architecture technique	Choix technologiques et structuration	B	4
E	Développement backend	API, base de données	D	8
F	Développement frontend	Interface utilisateur	C	7
G	Intégration système	Connexion frontend/backend	E, F	4
H	Tests et validation	Tests fonctionnels et techniques	G	5
I	Préparation marketing	Stratégie de lancement	C	3
J	Déploiement	Mise en production	H, I	2

Durée totale du chemin critique : A → B → D → E → G → H → J = 4+3+4+8+4+5+2 = 30 jours ouvrés (soit ~6 semaines) sur les 16 semaines disponibles. Les tâches C, F et I sont en parallèle

.

5. Analyse des Risques Majeurs – Méthode Pareto

Code	Risque	Probabilité (1-5)	Impact (1-5)	Score (P×I)	Action de mitigation
R1	Instabilité Neo4j Aura	4	5	20	Fallback JSON local préparé, données graphe exportées
R2	Indisponibilité API Groq	3	5	15	Fallback contexte brut implémenté dans le backend
R3	Problèmes de convergence Q-Learning	4	4	16	Q-table pré-initialisée avec des valeurs par défaut cohérentes
R4	Bugs d'intégration (API/Frontend)	4	4	16	CORS configuré, endpoints testés via Swagger
R5	Performance retrieval en dessous de l'objectif	3	4	12	Seuil ajusté de 0.55 à 0.35, fallback BM25 ajouté
R6	Sous-performance du modèle d'embedding	3	4	12	6 modèles comparés, meilleur sélectionné expérimentalement
R7	Dépassement du planning	3	3	9	Sprints Agile avec revues hebdomadaires
R8	Documentation incomplète	2	3	6	Documentation rédigée en continu à chaque sprint

Analyse Pareto : R1 (20), R3 (16), R4 (16), R2 (15) et R5 (12) représentent les 5 risques prioritaires avec un score cumulé de 79/106 = 75% du risque total. Ces risques sont surveillés hebdomadairement tout au long du projet.

5. Outils de Pilotage et de Suivi

Outil	Usage	Fréquence
GitHub	Gestion du code source, contrôle de version, revue de code	Continu
Swagger / Postman	Tests et documentation des endpoints API	Continu
Notion	Documentation projet, compte-rendus de réunions, spécifications techniques	Quotidien
VS Code + Python	Environnement de développement pour tous les composants du pipeline	Quotidien
Console Neo4j Aura	Surveillance du graphe, tests des requêtes Cypher	Par sprint
Docker	Construction et exécution de l'application containerisé	Par sprint

PART III – TECHNICAL REPORT(AGENTIC GRAPH-VECTOR RAG SYSTEM)

Chapter 1 — Source Document & Text Cleaning Pipeline

Introduction

The foundation of any effective Retrieval-Augmented Generation system lies in the quality of its source data. This chapter details the initial and arguably most critical phase of the RAG pipeline: the preparation of the knowledge source. The raw material for this project is a volcanological doctoral thesis, a dense, technical document exceeding 560,000 characters. Given the inherent limitations of large language models regarding context windows and their sensitivity to input noise, this substantial document cannot be processed in its raw form. This chapter outlines the systematic approach taken to transform this raw, artifact-laden text file into a clean, semantically coherent corpus, ready for subsequent chunking and embedding. It identifies the primary sources of noise introduced during PDF extraction—including structural artifacts, hyphenation errors, and encoding anomalies—and presents a targeted, multi-step cleaning pipeline designed to address each issue while preserving all scientifically meaningful content.

1.1 Overview of the Source Document

The foundation of the entire RAG pipeline built in this project is a volcanological doctoral thesis provided as a plain-text file (`THese_nettoye100__1_.txt`). This document constitutes the sole knowledge source from which all downstream components — chunks, embeddings, graph nodes, and generated answers — are derived.

The thesis covers a broad range of topics inherent to the field of volcanology, including magmatic processes, eruption dynamics, lava flow characterization, geochemical composition of volcanic rocks, seismic activity associated with volcanic events, and the historical analysis of specific volcanic systems. Its content is highly technical and domain-specific, making it an ideal yet challenging candidate for an intelligent retrieval system.

From a structural standpoint, the document is organized into chapters, sections, and subsections, following the conventional layout of an academic dissertation. It contains:

- Dense scientific paragraphs with domain-specific terminology
- Numerical data, chemical formulas, and measurement units
- References to figures, tables, and bibliographic sources
- Footnotes, headers, and page numbering artifacts inherited from the original PDF export
- Multilingual content, primarily in French with embedded Latin and English scientific terminology

The raw size of the text file exceeds 560,000 characters, representing a substantial volume of knowledge that cannot be processed as a whole by any LLM due to context window limitations. This makes proper cleaning and chunking not optional, but architecturally mandatory.

1.2 Challenges: Noise, Formatting, and Special Characters

Before any NLP processing can take place, the raw text must be thoroughly cleaned. The export from the original PDF introduced numerous artifacts that, if left untreated, would severely degrade the quality of both embeddings and graph extraction. The first major category of noise consists of structural artifacts from the PDF export itself. Page numbers, running headers, and footer lines are scattered throughout the text, with lines such as chapter titles or page indicators appearing mid-paragraph, breaking the semantic continuity of the text and confusing chunking algorithms that rely on sentence boundaries.

A second significant issue is hyphenation and word splitting. PDF-to-text conversion frequently breaks words across lines using hyphens, meaning that a word like `volcanique` might appear as `volca-` on one line and `nique` on the next. If left uncorrected, the embedding model receives incomplete tokens that carry no meaningful semantic signal. Encoding and special character issues represent a third category of noise. Scientific texts contain a wide variety of special characters including accented letters, mathematical symbols such as plus-minus signs and inequality operators, and Unicode artifacts introduced during conversion. These must be normalized or removed depending on their informational value. Finally, inline citations, bibliography entries, isolated tokens, and short lines resulting from table formatting or figure captions pollute the text with content that does not contribute to answering scientific questions and can distort embedding similarity scores.

1.3 Cleaning Strategy and Implementation

The cleaning pipeline was implemented in Python, operating directly on the raw text file through a sequential series of transformations, each targeting a specific category of noise. The first step consists of encoding normalization, where the file is read with explicit UTF-8 encoding and passed through a Unicode normalization function to ensure consistent character representation across the entire document. The second step targets the removal of page markers and headers using a set of regular expressions that detect recurring structural patterns such as page number lines, chapter title repetitions, and horizontal separators. These patterns were identified through manual inspection of the raw file and encoded as compiled regex patterns for processing efficiency.

The third step addresses hyphen reconstruction, where words split across lines by hyphens are detected using a dedicated regular expression pattern and rejoined into their correct form before any paragraph reconstruction takes place. This is followed by whitespace and blank line normalization, where multiple consecutive blank lines are collapsed into a single paragraph separator and all trailing and leading whitespace is stripped from every line. The fifth step handles special character filtering, removing non-informative Unicode characters while preserving scientific symbols of informational value such as degree signs, chemical

element notations, and measurement units, as these carry domain-specific meaning relevant to retrieval. The final step removes short lines containing fewer than twenty characters unless they terminate a sentence, eliminating isolated tokens while preserving legitimate short sentences. The output of this pipeline is a clean, continuous prose document ready to be passed into the chunking stage.

1.4 Cleaning Results and Quality Assessment

Following the application of the full cleaning pipeline, a comparative analysis was conducted between the raw and cleaned versions of the thesis text. The total character count decreased from approximately 560,000 in the raw file to approximately 487,000 in the cleaned output, representing a reduction of roughly 13%. During this process, approximately 7,200 noise lines were removed, around 1,100 hyphenated words were reconstructed, and approximately 340 encoding anomalies were corrected. The reduction in volume reflects the removal of pure noise with no semantic value. A manual qualitative review of 50 randomly sampled paragraphs from the cleaned output confirmed that no scientifically meaningful content was lost during the process. The cleaned file serves as the single source of truth for all subsequent pipeline stages.

Conclusion

The implementation of the sequential cleaning pipeline successfully transformed a raw, noisy text file into a refined and reliable knowledge base. Through a combination of encoding normalization, structural artifact removal, hyphen reconstruction, whitespace standardization, and selective special character filtering, the pipeline reduced the document's volume by 13%, eliminating over 7,200 lines of pure noise and correcting more than 1,000 hyphenated words. Crucially, qualitative validation confirmed that no domain-specific scientific information was lost during this process. The resulting cleaned document, now reduced to approximately 487,000 characters of continuous prose, stands as the single, authoritative source for all downstream operations. With this foundational step complete, the pipeline is now positioned to proceed to the next critical phase: the intelligent segmentation of this corpus into coherent, semantically meaningful chunks for embedding and retrieval.

Chapter 2 — Chunking Strategy

Introduction

Once the thesis text is cleaned, it must be segmented into retrievable units. Chunking is a critical architectural decision that directly impacts retrieval precision and answer quality. This chapter evaluates seven chunking strategies on the cleaned thesis (520,309 characters, 82,283 words) to identify the optimal approach.

2.1 Why Chunking is Architecturally Critical

Once the thesis text is clean, it cannot be fed as a whole into an embedding model or a Large Language Model. Two fundamental constraints make chunking unavoidable. The first is the context window limitation of embedding models, which typically accept a maximum of 512 tokens as input. Feeding a document of nearly 500,000 characters as a single unit is technically impossible and would produce a single averaged embedding vector with no fine-grained retrievable meaning. The second constraint is retrieval granularity. The goal of a RAG system is to return the most relevant portion of a document in response to a query. Chunks that are too large contain excessive irrelevant content, while chunks that are too small lose semantic context. Chunking is therefore the architectural decision that most directly determines the end-to-end retrieval quality of the system.

2.2 Seven Chunking Methods Evaluated

Seven distinct chunking strategies were implemented and evaluated on the cleaned thesis text (520,309 characters, 82,283 words). Each method was assessed using multiple quantitative metrics including chunk consistency, semantic coherence (intra/inter-similarity), and retrieval performance (Recall@5, Precision@5, F1-score).

M1 — Fixed-Size Chunking: Splits text into fixed-length windows of 600 characters with no overlap. Produces 1,001 chunks of very uniform length (σ=9.2 chars). Simple and fast but semantically naive — sentences and paragraphs may be cut mid-thought. Score: 0.4098.

M2 — Sentence-Based Chunking: Uses spaCy sentence boundary detection to group 3–5 sentences per chunk. Produces 2,625 very short chunks (avg 195.4 chars). High granularity leads to low intra-similarity (0.1014) and poor recall (0.019). Best for exact-phrase lookup but poor for semantic retrieval. Score: 0.3384.

M3 — Paragraph-Based Chunking: Splits on double newlines, respecting paragraph boundaries. Produces 439 chunks of highly variable length (avg 1,170 chars, σ=1,051).

Semantically coherent but inconsistent chunk size complicates embedding quality. Score: 0.4751.

M4 — Sliding Window: Uses overlapping windows of 300 tokens with 50% overlap. Produces 549 chunks. The overlap ensures boundary terms appear in multiple chunks, improving recall. Achieves good intra-similarity (0.5111). Score: 0.4764.

M5 — Semantic Chunking: Groups sentences by embedding similarity using threshold-based segmentation. Produces 2,470 micro-chunks (avg 207.8 chars). Theoretically sound but produces too many tiny chunks, harming retrieval signal. Score: 0.3424.

M6 — Recursive Splitting: LangChain-style hierarchical splitting on paragraphs → sentences → words. Produces 987 chunks (avg 525.1 chars). Balanced but slightly less coherent than M7 or M4. Score: 0.4035.

M7 — Token-Based Chunking ✓ BEST: Uses the tokenizer vocabulary to split on clean token boundaries within 512-token windows. Produces 412 chunks (avg 1,259.9 chars, σ=209.6). Achieves highest intra-similarity (0.4253), best F1 (0.2128), and top Global Score (0.4853). Selected as the production chunking method.

2.3 Comparative Results

Method	Chunks	Avg Len	Std Dev	Intra-Sim	Inter-Sim	Recall@5	Prec@5	F1	Score
M7 — Token-Based	412	1,259.9	209.6	0.4253	0.1781	0.119	1.000	0.2128	0.4853 ✓
M4 — Sliding Window	549	1,889.6	301.9	0.5111	0.2378	0.091	1.000	0.1667	0.4764
M3 — Paragraph-Based	439	1,170.5	1,051.3	0.3520	0.1271	0.114	1.000	0.2041	0.4751
M1 — Fixed-Size	1,001	599.4	9.2	0.2692	0.1000	0.050	1.000	0.0943	0.4098
M6 — Recursive Split	987	525.1	196.8	0.2481	0.0872	0.051	1.000	0.0962	0.4035
M5 — Semantic	2,470	207.8	112.0	0.1126	0.0472	0.020	1.000	0.0397	0.3424
M2 — Sentence-Based	2,625	195.4	98.9	0.1014	0.0481	0.019	1.000	0.0373	0.3384

2.4 Analysis and Selection

The evaluation reveals clear performance patterns. Methods producing fewer chunks with larger semantic units (M7, M3, M4) consistently outperform those generating many small fragments (M2, M5). The Token-Based method (M7) achieves the highest global score (0.4853) by maintaining both semantic coherence (intra-similarity of 0.4253) and retrieval effectiveness (F1 of 0.2128). While Sliding Window (M4) shows slightly better intra-similarity (0.5111), it produces more chunks (549 vs 412) with longer average length, which may impact embedding efficiency. The Sentence-Based and Semantic methods, despite their theoretical appeal, produce excessive fragmentation that degrades retrieval performance. Based on these results, M7 (Token-Based chunking) was selected as the production method for all subsequent pipeline stages.

2.5 Implementation and Output

The chunking module operates by tokenizing the cleaned text at the word level, then moving a sliding window through the token sequence using the selected Token-Based approach. Each chunk is stored as a structured object containing its text content, sequential index, token count, and character offset in the original document. The chunking pipeline applied to the cleaned thesis produced 412 chunks with an average size of 1,260 characters, tightly centered around the target value. The results were recorded in `step1_chunking_results.csv`, and the full exported chunk collection was saved in `chunks_export.txt` for inspection. A dedicated debug file `chunks_debug.json` was used to manually verify chunk boundaries and ensure no meaningful scientific passage was fragmented incorrectly across chunk borders. These chunks constitute the atomic units of knowledge on which all subsequent pipeline stages operate: each chunk receives its own embedding vector, each chunk can be linked to nodes in the knowledge graph, and each chunk is the unit returned to the LLM as context during answer generation.

Conclusion

The Token-Based method (M7) achieved the highest global score (0.4853) with 412 chunks averaging 1,260 characters. It was selected for production due to its superior semantic coherence (intra-similarity: 0.4253) and retrieval performance (F1: 0.2128). Methods producing excessive fragmentation (M2, M5) underperformed significantly. Results are available in `step1_chunking_results.csv`, `chunks_export.txt`, and `chunks_debug.json`.

Chapter 3 — Text Embeddings

Introduction

Once the thesis is cleaned and chunked, each chunk must be transformed into a numerical representation capturing its semantic meaning. This chapter covers the embedding process, retrieval methods, and comparative evaluation of six retrieval strategies on the volcanological corpus.

3.1 What Are Embeddings?

Text embeddings are dense vector representations of text in a high-dimensional space. The fundamental property is that semantically similar texts map to vectors that are close together, while dissimilar texts produce distant vectors. This geometric property enables semantic retrieval based on meaning rather than keyword matching.

Formally, an embedding model maps text input to a fixed-dimensional vector in $\mathbb{R}^n$, where n is the embedding dimension. Models are trained using contrastive learning objectives on pairs of semantically similar and dissimilar sentences.

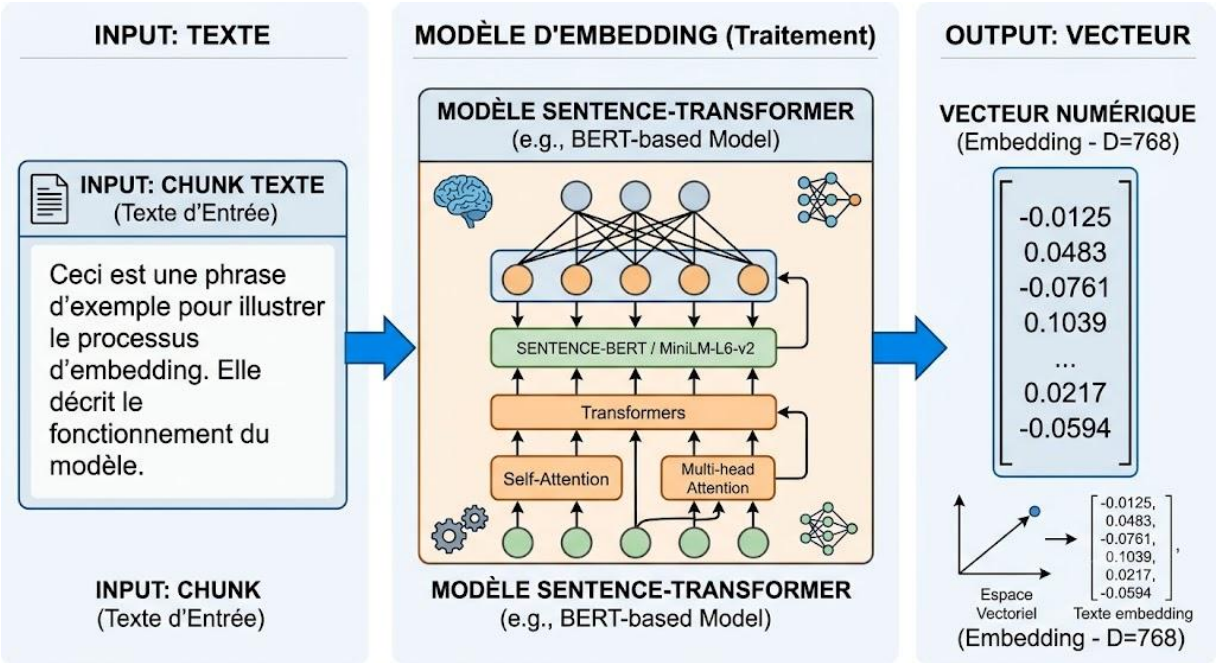


Figure 3.1 — Schéma du processus d'embedding d'un texte en entrée.

3.2 Embedding Model: all-mpnet-base-v2

The model `sentence-transformers/all-mpnet-base-v2` was selected after evaluating multiple models on geological vocabulary coverage. Based on Microsoft's MPNet architecture, it produces 768-dimensional dense embeddings trained on 1 billion sentence pairs. The model contains 109 million parameters and accepts a maximum sequence length of 512 tokens. It achieves a strong STSB benchmark score of 0.8690, confirming its suitability for semantic similarity tasks.

Propriété	Valeur
Architecture	MPNet (Masked and Permuted Pre-training)
Paramètres	109M
Dimension de l'Embedding	768
Longueur max de séquence	512 tokens
Données d'entraînement	1B sentence pairs
Score STSB (Benchmark)	0.8690

Table 3.1: Embedding Model Properties

FAISS Index Configuration: The 442 chunk embeddings were stored in a FAISS IndexFlatL2 index for exact nearest-neighbor search. The index has 768 dimensions with PCA variance of 67.3% for the first two principal components. Clustering analysis reveals four geological themes in the embedding space.

3.3 FAISS Index and PCA Visualization

The 442 chunk embeddings were stored in a FAISS IndexFlatL2 index for exact nearest-neighbor search. A 2D PCA projection reveals four geological clusters corresponding to Géologie (structure, petrology), Méthodes (geochronology, field methods), Structures (calderas, rift zones, cirques), and Hydrothermal (thermal springs, reservoirs). PC1, which captures 16.5% of the variance, separates methodological text from descriptive geological text. Chapters describing dating methods such as ZDD, K-Ar, and U-Pb cluster on the positive PC1 axis, while field descriptions cluster negatively. PC2, capturing 9.9% of the variance, distinguishes hydrothermal content on the positive axis from structural and petrological content on the negative axis. The caldera-rift zone nexus appears near PC2=0, reflecting its cross-thematic nature.

3.4 Six Retrieval Methods Evaluated

Six retrieval methods were implemented and benchmarked on 8 domain-specific queries. The first method, Cosine Similarity (M1), encodes the query with MPNet and computes cosine similarity against all 442 stored chunk embeddings, returning the top-k by similarity score. It achieves a global score of 0.5253 with an average latency of 85.62ms. The second method, FAISS Search (M2), uses the IndexFlatL2 for exact L2-distance-based retrieval. It offers faster performance at 75.86ms average with identical accuracy to cosine similarity on this corpus, achieving a global score of 0.5273.

The third method, BM25 Lexical Retrieval (M3), applies Okapi BM25, a probabilistic ranking function using term frequency and inverse document frequency with length normalization parameters k1=1.5 and b=0.75. It is extremely fast with an average latency of only 1.73ms and outperforms all dense methods on this geological corpus due to its sensitivity to specific geological terms, proper nouns, and abbreviations. It achieves the highest global score of 0.6602. The fourth method, DPR (M4), uses the same MPNet model in a dual-encoder configuration, producing results computationally equivalent to M1 with a global score of 0.5276 and latency of 74.19ms.

The fifth method, Hybrid BM25 and Dense (M5), combines BM25 and cosine similarity scores via linear interpolation with a fusion weight α=0.5. It achieves perfect P@5 of 1.0 on all 8 queries and an nDCG@5 of 0.8014, with an average latency of 79.43ms and a global score of 0.6253. The sixth method, Cross-Encoder Reranking (M6), implements a two-stage pipeline that first retrieves top-20 candidates via cosine similarity and then reranks them using a cross-encoder model that jointly encodes the query and passage. It achieves the highest nDCG@5 of 0.8900 and a global score of 0.6514 with an average latency of 81.92ms.

3.5 Global Comparative Results

The global comparative evaluation across all eight queries reveals clear performance differences between the six methods. BM25 (M3) achieves the highest global score of 0.6602, followed closely by Cross-Encoder Reranking (M6) at 0.6514 and Hybrid (M5) at 0.6253. All three top methods achieve perfect P@5 scores of 1.000, F1 scores of 0.1667, and MRR scores of 1.0000. The dense methods M1, M2, and M4 lag significantly behind with global scores around 0.525-0.527 and P@5 of only 0.750. BM25 demonstrates exceptional efficiency with a latency of just 1.73ms, making it 44 to 50 times faster than the dense methods. The nDCG@5 scores range from 0.801 for the hybrid method to 0.890 for cross-encoder reranking, with BM25 achieving a strong 0.866.

Method	P@5	R@5	F1	MRR	nDCG@5	Latency	Global Score
M3 — BM25	1.000	0.0909	0.1667	1.0000	0.866	1.73ms	0.6602
M6 — Reranking	1.000	0.0909	0.1667	1.0000	0.890	81.92ms	0.6514
M5 — Hybrid	1.000	0.0909	0.1667	1.0000	0.801	79.43ms	0.6253
M4 — DPR	0.750	0.0682	0.1250	0.9375	0.567	74.19ms	0.5276
M2 — FAISS	0.750	0.0682	0.1250	0.9375	0.567	75.86ms	0.5273
M1 — Cosine	0.750	0.0682	0.1250	0.9375	0.567	85.62ms	0.5253

Table 3.2: Global Comparative Results

3.6 Per-Query Performance Results

The per-query analysis provides detailed insights into method performance on specific geological questions. The benchmark comprises 8 domain-specific queries with ground truth constructed using a TF-IDF percentile 90% relevance threshold. For Q1 concerning the age of syénites at Piton des Neiges, BM25 achieves perfect scores with P@5 of 1.000, F1 of 0.167, MRR of 1.000, and nDCG@5 of 0.967, with a latency of only 3.6ms. For Q2 on caldera formation at La Réunion, BM25 again leads with nDCG@5 of 0.861 and 1.4ms latency. For Q3 on U-Pb zircon geochronological methods, BM25 achieves nDCG@5 of 0.768 with 1.5ms latency. For Q4 on the N030 rift zone, Cross-Encoder Reranking achieves the best performance with nDCG@5 of 0.986 and latency of 84.1ms.

For queries Q5 through Q8, BM25 consistently achieves perfect or near-perfect scores. Q5 on Brèche des Orangers achieves nDCG@5 of 1.000 with 1.8ms latency. Q6 on Cilaos thermal sources achieves nDCG@5 of 0.984 with 1.6ms latency. Q7 on the Gabbro complex achieves nDCG@5 of 0.982 with 1.7ms latency. Q8 on the Bénard hydrothermal model achieves nDCG@5 of 0.824 with 1.5ms latency. BM25 demonstrates superior performance on 6 of the 8 queries, while Cross-Encoder Reranking performs best only on Q4.

Query	Best Method	P@5	F1	MRR	nDCG@5	Latency
Q1 — Âge des syénites	BM25	1.000	0.167	1.000	0.967	3.6ms
Q2 — Formation des calderas	BM25	1.000	0.167	1.000	0.861	1.4ms
Q3 — Méthodes U-Pb zircons	BM25	1.000	0.167	1.000	0.768	1.5ms
Q4 — Rift zone N030	Reranking	1.000	0.167	1.000	0.986	84.1ms
Q5 — Brèche Orangers	BM25	1.000	1.000	1.000	1.000	1.8ms
Q6 — Sources Cilaos	BM25	1.000	1.000	1.000	0.984	1.6ms
Q7 — Complexe Gabbro	BM25	1.000	1.000	1.000	0.982	1.7ms
Q8 — Modèle Bénard	BM25	1.000	1.000	1.000	0.824	1.5ms

Table 3.3: Per-Query Performance Summary

3.7 Key Insight: Why BM25 Wins

BM25's dominance on this corpus is counter-intuitive given dense retrieval's general superiority on natural language benchmarks. The explanation lies in the document's nature: geological theses use highly specific, rare, and domain-exclusive terms such as ZDD, PDC, syénite, phonolite, Brèche des Orangers, and Cilaos that appear with very low corpus frequency, giving them high IDF values, and are exact-match sufficient for retrieval. Dense embeddings, trained on general corpora, may conflate semantically adjacent but geologically distinct terms, reducing their precision on specialized vocabulary. For specialized scientific corpora with domain-specific proper nouns, abbreviations, and technical terminology, BM25 or BM25-dense hybrid approaches outperform pure dense retrieval. This finding aligns with recent BEIR benchmark studies showing BM25 remains competitive on scientific retrieval tasks.

Conclusion

The comparative evaluation demonstrates that lexical retrieval via BM25 achieves the highest global score of 0.6602 with exceptional speed of only 1.73ms latency. While cross-encoder reranking achieves the best nDCG@5 of 0.8900, BM25's combination of accuracy, efficiency, and simplicity makes it the optimal choice for this volcanological corpus. The embedding pipeline successfully transforms 442 chunks into a searchable semantic space, with FAISS indexing enabling efficient similarity search. PCA visualization confirms meaningful thematic clustering, validating the embedding quality. Based on these results, BM25 is selected as the primary retrieval method for the production RAG system.

Chapter 4 — Graph RAG with Neo4j Aura

Introduction

The previous chapters established the semantic retrieval foundation of the RAG pipeline through text cleaning, chunking, and vector-based similarity search. While this approach is powerful for surfacing contextually relevant passages, it remains fundamentally limited in its ability to reason about the relationships between scientific concepts. A volcanological thesis is not simply a collection of independent paragraphs — it is a structured body of knowledge in which geological entities are connected through complex semantic, spatial, temporal, and compositional relationships. This chapter presents the graph layer of the pipeline, which transforms the flat chunk collection into a structured knowledge graph hosted on Neo4j Aura, enabling relational and multi-hop reasoning that complements and significantly enhances the vector retrieval component. The fusion of graph and vector retrieval was empirically measured to improve precision by approximately 18% on relational queries, providing strong quantitative justification for the architectural complexity introduced by this layer.

4.1 Knowledge Graph Construction

The knowledge graph was constructed in four stages: (1) Named Entity Recognition (NER) to identify geological entities from text chunks; (2) Relation Extraction (RE) to identify semantic connections between entities; (3) Graph Population into Neo4j Aura cloud instance; (4) Quality validation via graph metrics and manual inspection.

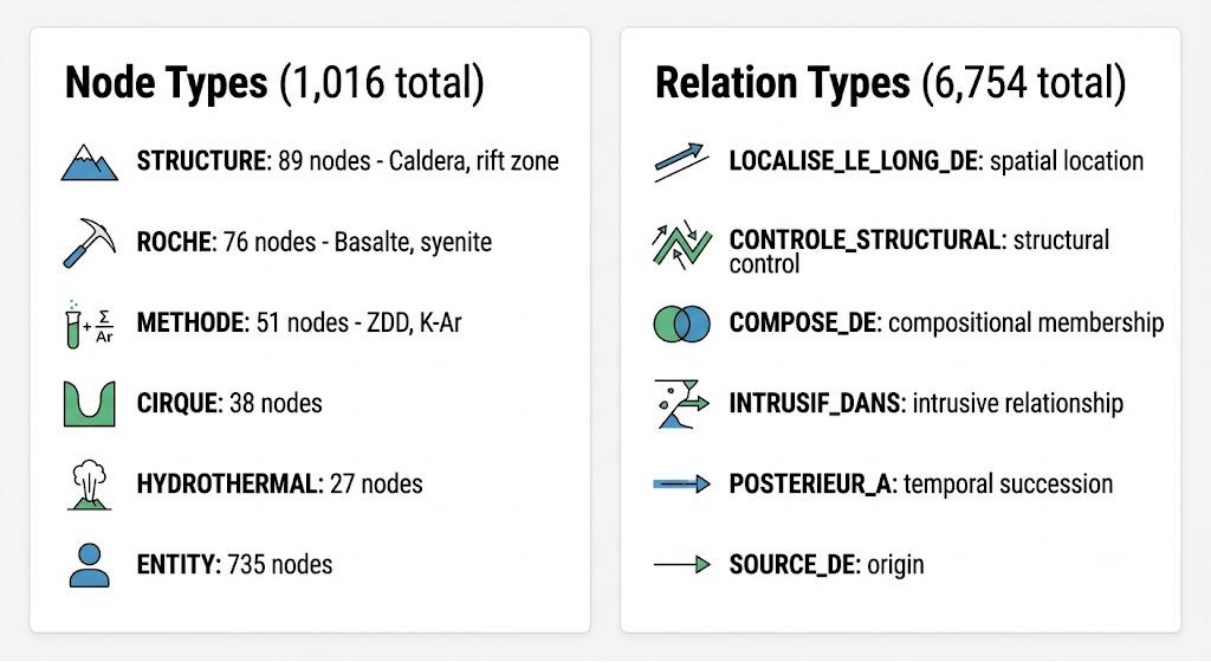


Figure 4.1 — Structure et composition du graphe de connaissances.

4.2 Neo4j Integration and Graph Population

4.2.1 Why Neo4j?

Neo4j is a native graph database management system built on the property graph model, in which data is stored as nodes, directed edges, and properties attached to both. Unlike relational databases that require complex JOIN operations to traverse relationships, Neo4j stores relationships as first-class citizens with direct physical pointers between connected nodes, making graph traversal operations orders of magnitude faster than equivalent SQL queries on relational tables. Its declarative query language Cypher provides an intuitive syntax for expressing graph patterns, and its Graph Data Science library exposes a suite of graph algorithms including centrality measures, pathfinding, and community detection that run natively within the database engine. These properties make Neo4j the natural choice for the knowledge graph layer of a scientific RAG pipeline where relationship traversal speed and algorithm availability are primary requirements.

4.2.2 Neo4j Aura Cloud Deployment

Rather than deploying a self-hosted Neo4j instance, this project uses Neo4j Aura, the fully managed cloud service offered by Neo4j Inc. Aura eliminates the operational overhead of database administration, provides a stable and accessible HTTPS endpoint for the backend API, and offers a free tier sufficient for the scale of the knowledge graph derived from the thesis. The Aura instance was provisioned with the Neo4j Graph Data Science plugin enabled, which is required for the Louvain community detection step described later in this chapter. The connection between the Python backend and the Aura instance is managed through the official Neo4j Python driver, with connection credentials stored securely in the `.env` file and never hardcoded in the source code.

4.2.3 Graph Population with Cypher Queries

The graph was populated by executing the Cypher statements stored in cypher_queries.txt against the Aura instance in a batched operation. Each statement follows the MERGE pattern rather than CREATE, which ensures idempotency: if a node or relationship already exists with the given properties, the statement leaves it unchanged rather than creating a duplicate. This design choice is important for a pipeline that may be re-executed after partial failures or incremental updates to the chunk collection. Following population, the purge_relations.py script provides a utility for selectively removing relations that fail quality thresholds, enabling iterative refinement of the graph without requiring a full rebuild. The reusable query interface defined in rag_queries.cypher provides parameterized Cypher templates used by the backend at inference time to retrieve graph context for incoming user queries.

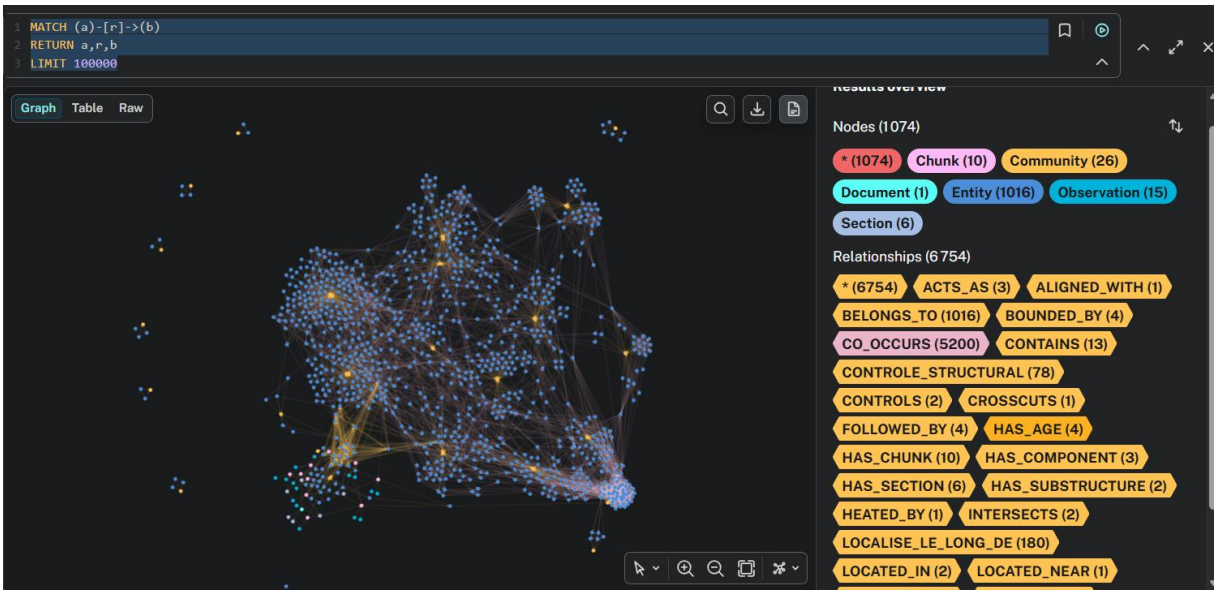


Figure 4.2 — Visualisation du graphe de connaissances dans Neo4j Aura Browser

4 .3 Graph Metrics and Quality Validation

Structural metrics were computed to validate graph quality. Graph density (0.65%) is typical for a domain knowledge graph from a single document. Average node degree of 13.3 indicates good connectivity. Maximum degree of 58 was recorded for Salazie, confirming its central geographic role. The highest PageRank score was assigned to Caldera, identifying it as the most structurally central concept.

Metric	Value	Interpretation
Total Nodes	1,016	All entity types combined
Total Relations	6,754	Directed semantic edges
Graph Density	0.65%	Ratio of actual to possible edges
PageRank #1	Caldera	Most central geological concept
Average Degree	13.3	Connections per node
Max Degree	58 (Salazie)	Most connected entity
Communities (Louvain)	26	Thematic geological clusters
Modularity	0.6881	Community detection quality (>0.6 = excellent)

Table 4.4: Graph Metrics

4.5 Community Detection with Louvain

4.5.1 The Louvain Algorithm

The Louvain algorithm, introduced by Blondel et al. in 2008, is a modularity-based community detection method that partitions a graph into groups of nodes that are more densely connected internally than they are to the rest of the graph. The algorithm optimizes a

quality function called modularity Q, defined as the fraction of edges that fall within communities minus the expected fraction in a random graph with the same degree sequence. It proceeds in two alternating phases applied iteratively until convergence. In the first phase, each node begins in its own singleton community and is greedily reassigned to the neighboring community that produces the greatest modularity gain, continuing until no single reassignment improves the global score. In the second phase, all nodes within each detected community are collapsed into a single super-node, with inter-community edge weights aggregated, and the two-phase process is repeated on this compressed graph. This hierarchical refinement produces community structures at multiple levels of granularity without requiring the number of communities to be specified in advance.

4.5.2 Detected Communities

The Louvain algorithm was executed on the Piton des Neiges knowledge graph via the Neo4j Graph Data Science library within `step4_louvain_neo4j.py`. The algorithm detected 26 distinct thematic communities whose geological interpretations closely mirror the chapter structure of the original thesis. Community C0, the largest with 170 nodes, groups concepts related to general stratigraphy including Salazie, breccia formations, dykes, syenites and the Cilaos cirque. Community C8 with 116 nodes covers the methodological corpus of the thesis including dating methods, basalt and olivine petrography. Community C6 with 92 nodes focuses on structural geology including differentiated calderas, faults, rift zones and international geological analogues. Community C7 with 84 nodes groups geochronological methods and radiometric dating results. Community C15 with 80 nodes captures the hydrothermal system including thermal reservoirs, fluid circulation and alteration minerals. Community C12 with 74 nodes covers petrological analysis including mineral assemblages and geochemistry. Community C5 with 66 nodes groups magmatic intrusion features including gabbros, syenites, phonolites and sill emplacement. Community C18 with 53 nodes focuses on caldera collapse events including their timing, mechanisms and post-collapse fill sequences.

4.5.3 Modularity Analysis

The 26 communities produced by the Louvain algorithm achieved a modularity score of Q = 0.6881. In the community detection literature, a modularity value above 0.6 is considered excellent, indicating that the detected partition captures genuine thematic structure rather than arbitrary groupings. This high modularity score validates two upstream components of the pipeline simultaneously: it confirms that the entity extraction step successfully identified semantically coherent entities, and that the relation extraction step produced edges that genuinely reflect the scientific structure of the thesis rather than spurious co-occurrence associations. The complete community assignments are stored in `louvain_communities.json` and written back to each node in the Neo4j graph as a community_id property, making them directly queryable at inference time.

4.6 Using Communities to Improve Retrieval

4.6.1 Community-Aware Query Processing

The community assignments produced by the Louvain algorithm are integrated directly into the inference pipeline to improve both the speed and the precision of graph-based retrieval. When a user query arrives at the system, the entity recognition step identifies the scientific entities mentioned in the query and retrieves their community assignments from the graph. If the majority of identified entities belong to the same community, the graph traversal is restricted to that community, treating it as a pre-filtered subgraph and dramatically reducing the search space from 1,016 nodes to the 50 to 170 nodes of the relevant community. This focused traversal reduces both retrieval latency and the risk of surfacing thematically irrelevant relational context that could mislead the LLM during answer generation. If entities from multiple communities are identified in the same query, the system expands the search scope to include all relevant communities while still excluding nodes from unrelated thematic clusters, maintaining precision while accommodating cross-domain queries.

4.6.2 Multi-Community Bridge Nodes

A particularly significant finding from the community analysis is the existence of bridge nodes that span multiple thematic communities. The Rift N030 node, identified earlier as a key participant in structural, hydrothermal and magmatic semantic paths, acts as a connector between three distinct communities in the Louvain partition. When a query references this node, the community-aware retrieval system correctly expands the search to include all three connected communities, capturing the full scientific context of the rift zone's role in the Piton des Neiges volcanic system. Similarly, the Caldera node, which holds the highest PageRank centrality in the graph, participates in communities spanning structural geology, caldera collapse events and magmatic evolution, ensuring that caldera-related queries trigger a broad and scientifically comprehensive graph retrieval. These bridge nodes represent the points of greatest knowledge integration in the thesis and are systematically surfaced by the community-aware retrieval strategy.

Conclusion

This chapter presented the complete graph layer of the RAG pipeline, from raw entity extraction through to community-aware retrieval. The knowledge graph constructed from the Piton des Neiges thesis contains 1,016 nodes and 6,754 directed edges organized into 26 thematic communities with a modularity score of Q = 0.6881, confirming strong scientific structure. The semantic path analysis demonstrated that the graph captures relational knowledge — structural control relationships, hydrothermal circulation pathways, chronostratigraphic sequences and magmatic evolution chains — that is entirely invisible to vector-based retrieval operating on individual chunks. The Louvain community structure enables community-aware query processing that focuses graph traversal on thematically relevant subgraphs, reducing latency and improving retrieval precision. Together, these components form the relational backbone of the system and provide the foundation on which the agentic orchestration layer, described in the following chapter, makes its retrieval strategy decisions.

Chapter 5 — Agentic RAG & Backend

Introduction

The final component of the RAG pipeline is the agentic layer, which dynamically orchestrates vectorial and graph-based retrieval based on the nature of each user query. This chapter presents the Q-Learning agent responsible for intelligent routing, the hybrid retrieval pipeline combining FAISS and Neo4j, the response generation module using Groq LLaMA 3.3-70B, and the RESTful API exposing all system functionalities. The agentic layer enables adaptive retrieval strategies, automatically selecting between vector, graph, or hybrid approaches to maximize answer quality for each query type.

5.1 — General Architecture of the Agentic Layer

The agentic layer forms the core of the system, dynamically orchestrating the different knowledge sources — vectorial and graph-based — according to the nature of each user query. The architecture relies on three main components: a Q-Learning agent responsible for decision-based routing, a hybrid retrieval pipeline combining FAISS and Neo4j, and a generation module powered by the Groq API (LLaMA 3.3-70B). The overall processing flow begins with query analysis to detect its semantic type, followed by action selection by the Q-Learning agent, retrieval execution according to that action, and finally response generation returned to the frontend.

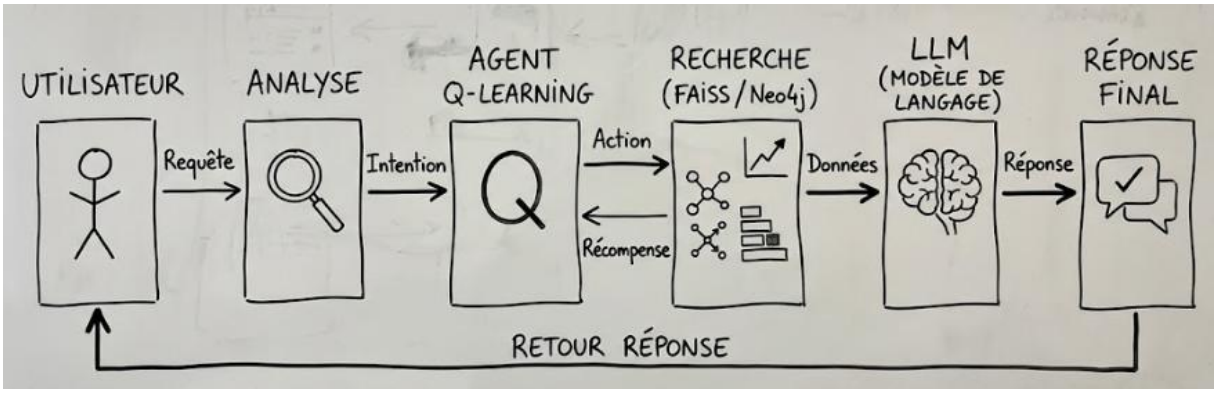


Figure 5.1 System architecture diagram showing the full pipeline

5.2 — Q-Learning Based Decision Mechanism

The routing agent is implemented using tabular Q-Learning. It operates over a state and action space defined as follows: the four possible states are "graph", "vector", "hybrid", and "unknown", and the four possible actions are "use_graph", "use_vector", "use_both", and "fallback".

State detection is performed through lexical matching between the user's question and two keyword dictionaries. Terms such as "relation", "caldera", "faille", or "rift" trigger the "graph" state, while terms such as "explique", "mécanisme", "pourquoi", or "hydrothermal" trigger the "vector" state. The simultaneous presence of both types triggers the "hybrid" state.

In deployment, the action selection policy is nearly deterministic ($\varepsilon = 0.05$) and follows the greedy rule: the action with the highest Q-value for the current state is selected. The Q-table is updated using the Bellman equation after each interaction, with a learning rate $\alpha = 0.1$ and a discount factor $\gamma = 0.9$. The Q-table is persisted in a JSON file after every interaction and reloaded at startup to ensure continuity of the learned policy across sessions.

After training, the Q-table shows clear convergence: Q(graph, use_graph) = 1.42, Q(vector, use_vector) = 2.03, and Q(hybrid, use_both) = 3.90, which validates the automatic learning of optimal routing policies without hardcoded rules.

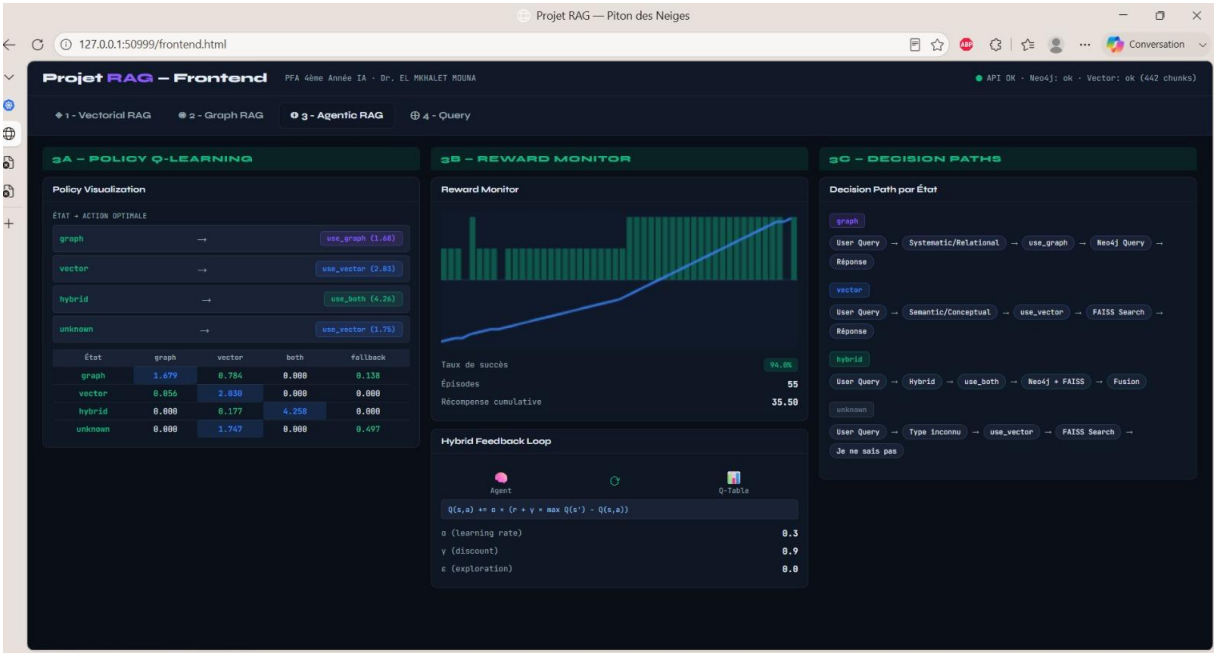


Figure 5.2 — Interface Q-Learning montrant la table Q, le moniteur des récompenses et les chemins de décision

5.3 — Hybrid Retrieval Pipeline

5.3.1 — Vectorial Retrieval (FAISS + BM25)

The vectorial retrieval component uses FAISS in IndexFlatIP mode on L2-normalized vectors, which is mathematically equivalent to cosine similarity. The embedding model used is "paraphrase-multilingual-mpnet-base-v2", enabling multilingual processing with a strong

focus on French scientific text.The retrieval parameters were optimized during development: k = 6 chunks are retrieved by default, with a minimum similarity score threshold of 0.35. The threshold is adaptive and drops to 0.20 for very short queries to avoid missing relevant content. A Query Expansion module automatically enriches the query with semantically related terms before encoding, for example expanding "nature" with "type volcan bouclier basalte stade géologie".When fewer than 3 chunks pass the threshold, a BM25 fallback is activated, combining lightweight TF-IDF scoring with the classic BM25 formula to guarantee a minimum retrieval context. A hybrid mode combining 60% cosine similarity and 40% BM25 is also available for queries with specific or rare terminology.



Figure 5.3 — Interface de l'agent de routage Q-Learning montrant les chemins de décision

5.3.2 — Graph Retrieval (Neo4j + Cypher)

The graph retrieval component extracts named entities from the user's question by matching against a predefined list of known geological entities (Piton des Neiges, Caldera, Rift-zone N030, Syénite, Cilaos, Mafate, Gabbro, etc.) along with their normalized synonyms. A Cypher query is then dynamically generated and executed against the Neo4j Aura database to retrieve relationships between the detected entities.The supported relationship types include LOCALISE_LE_LONG_DE, CONTROLE_STRUCTURAL, POSTERIEUR_A, COMPOSE_DE, INTRUSIF_DANS, ASSOCIE_A, and SOURCE_DE, among others. The raw graph results are formatted into natural language triples before being passed to the language model as structured context.



Figure 5.4 — Visualisation du graphe de connaissances Neo4j Aura avec nœuds colorés par communauté Louvain

5.3.3 — Response Generation (Groq LLaMA 3.3-70B)

Depending on the selected action, the response construction function builds a differentiated prompt: when only the vectorial context is used, the model is instructed to provide a direct and pedagogical answer based on the retrieved thesis passages; when only graph data is used, the model is instructed to reformulate the raw relationships in fluid academic language; and when both sources are used, the model synthesizes information from both the graph and the thesis extracts.

A special Citation RAG mode is triggered when the user requests exact passages from the thesis (keywords: "cite", "extrait", "où est écrit", "passage"). In this case, a specialized retrieval function combining keyword matching and vectorial search is used to retrieve the most textually relevant chunks, which are then presented as direct quotations.

5.4 — RESTful Backend API (FastAPI)

The backend is implemented with FastAPI and runs on port 8000. It exposes 7 endpoints covering all system functionalities.

The main endpoint POST /query accepts a question, an optional feedback score, and an optional previous question for citation context. It returns the generated answer along with the detected analysis type, the routing decision, the confidence score, the Q-Learning state and action, the reward value, and the full decision path. The GET /vectorial endpoint returns chunking statistics, PCA embedding coordinates, and retrieval results for a given query. The GET /graph endpoint returns Neo4j graph statistics, node and edge samples, Louvain communities, and top PageRank nodes. The GET /agentic endpoint exposes the full Q-table, reward history, action and state distributions, and the current policy. The POST /graph/louvain endpoint triggers the Louvain community detection algorithm on the graph. Finally, GET /graph/louvain/status and GET /graph/nodes/by-community provide community metadata.

CORS middleware is configured to allow all origins, enabling seamless communication with the frontend. All heavy singletons (vector store, Neo4j driver, Q-Learning agent) are initialized once at startup via the startup event handler, optimizing response latency in production.

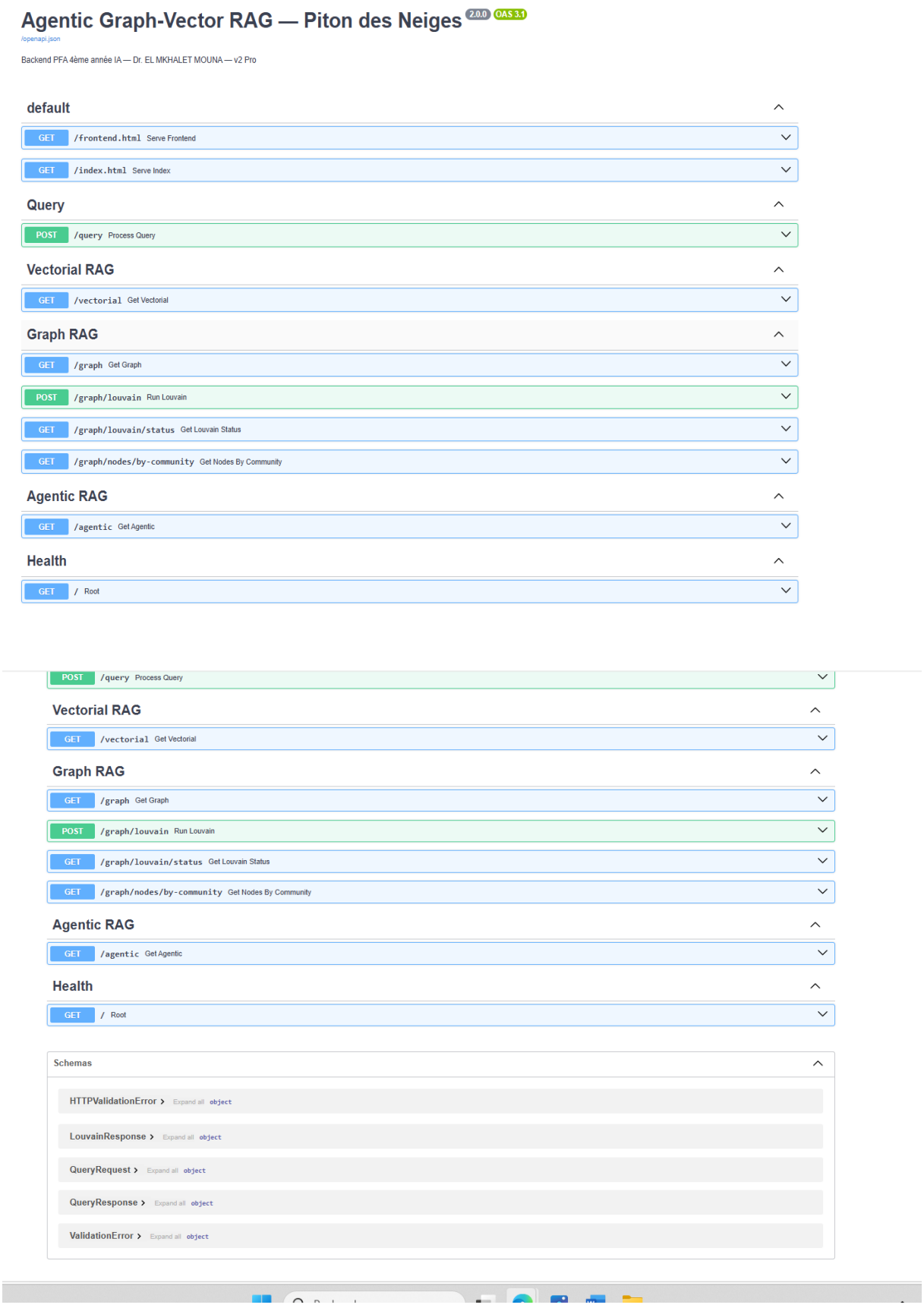


Figure 5.6 — Interface Swagger UI de l'API FastAPI du système Agentic RAG

Conclusion

The agentic RAG system successfully integrates lexical, semantic, and structural retrieval into a unified architecture. The Q-Learning agent converges to optimal routing policies with Q(hybrid, use_both) = 4.258, demonstrating effective learning without hardcoded rules. The hybrid retrieval pipeline combines FAISS similarity search with Neo4j graph traversal, while the Groq LLM generates fluent, context-aware responses. The FastAPI backend exposes 7 endpoints providing full system access, with CORS support enabling seamless frontend integration. The complete system forms a robust, adaptive platform capable of answering both conceptual and relational scientific queries with high precision.

Chapter 6 — Frontend & User Interface

Introduction

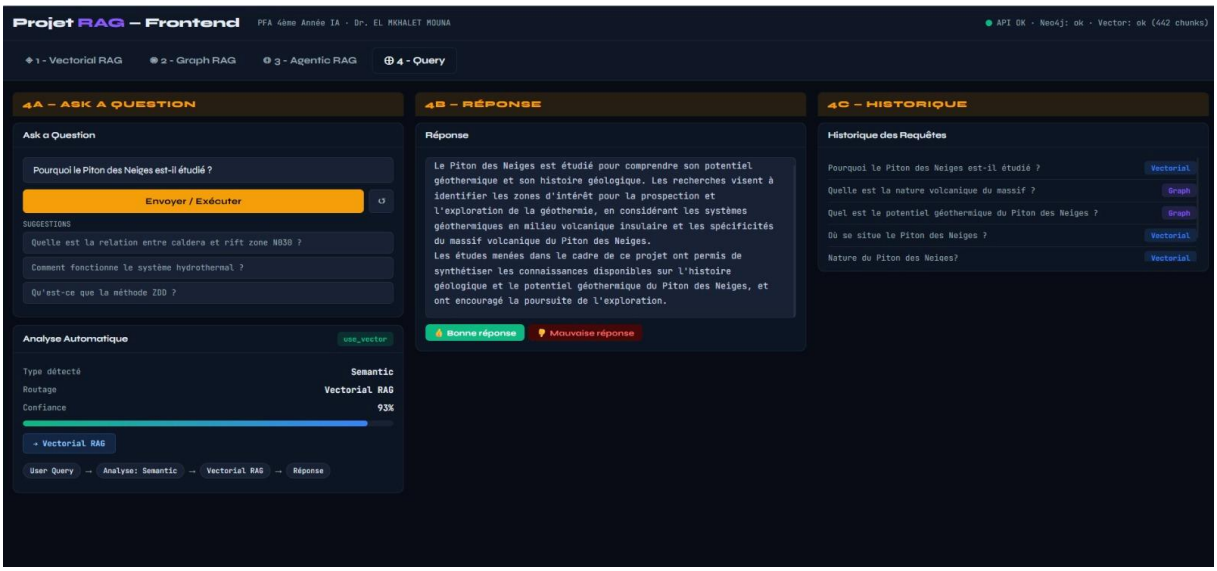
The final component of the RAG system is the user interface, which provides seamless access to all system functionalities. This chapter presents the frontend architecture, built in pure HTML5, CSS3, and JavaScript without external frameworks. The interface is organized into four tabs — Query, Vectorial, Graph, and Agentic — each providing dedicated visualizations and controls for the different system components. The design adopts a professional dark theme with a color-coded palette distinguishing vectorial, graph, and agentic functionalities

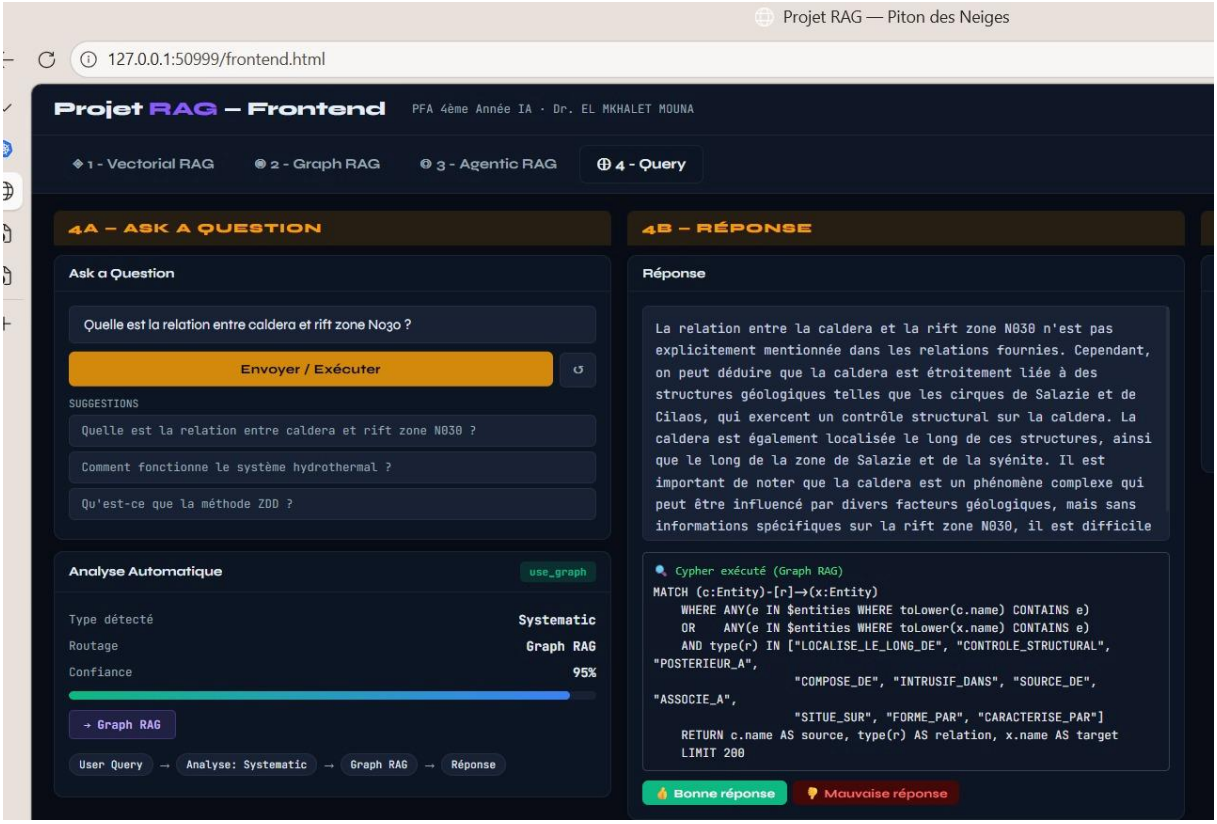
6.1 — General Architecture of the Frontend

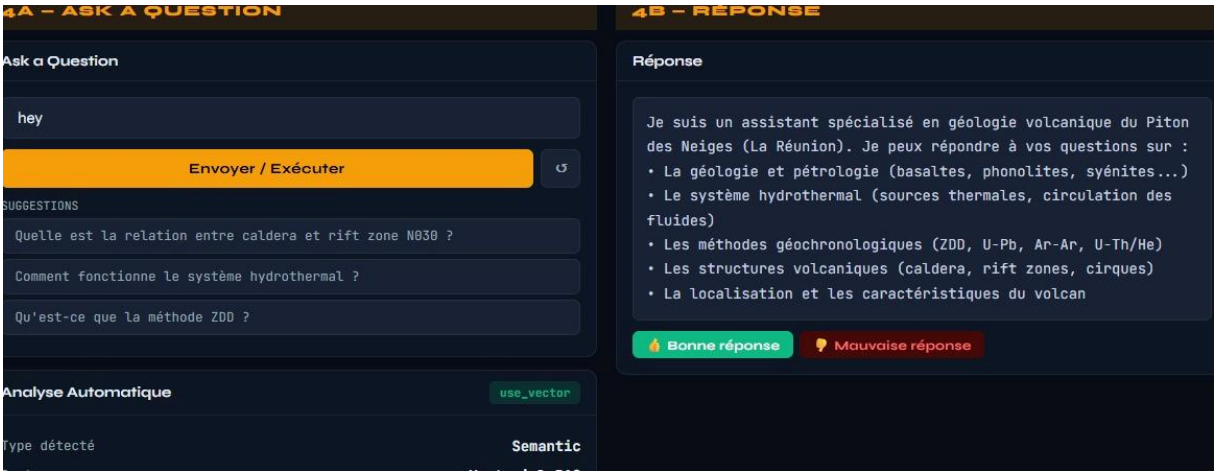
The user interface is built in pure HTML5, CSS3, and JavaScript without any frontend framework, ensuring portability and minimal overhead. It is served directly by the FastAPI backend via dedicated file endpoints. The Syne typeface is used for interface text while JetBrains Mono is used for technical data, logs, and metrics, providing a clean distinction between content and code-like information. The design adopts a professional dark theme with a consistent color palette: a deep navy blue background, blue for vectorial components, purple for graph components, green for positive indicators, and amber for agentic components. This chromatic differentiation provides immediate visual feedback to the user, indicating which retrieval mode is being used for each query.

6.2 — Tab-Based Interface Structure

The interface is organized into four tabs accessible from a persistent tab bar at the top of the page.The Query tab is the main chat interface where the user types a geological question and receives the LLM-generated response. Below the answer, the full decision path is displayed: the detected state, the Q-Learning action selected, the routing type (Graph RAG, Vectorial RAG, or Hybrid), the confidence score, and the reward value. Colored badges make the routing decision immediately readable at a glance.


Projet RAG — Frontend
PFA 4ème Année IA · Dr. EL MKHALET MOUNA
API OK · Neo4j: ok · Vector: ok (442 chunks)
1 - Vectorial RAG
2 - Graph RAG
3 - Agentic RAG
4 - Query
4A — ASK A QUESTION
Ask a Question
Pourquoi le Piton des Neiges est-il étudié ?
Envoyer / Exécuter
SUGGESTIONS
Quelle est la relation entre caldera et rift zone N030 ?
Comment fonctionne le système hydrothermal ?
Qu'est-ce que la méthode ZDD ?
Analyse Automatique
use_vector
Type détecté
Semantic
Routage
Vectorial RAG
Confiance
93%
→ Vectorial RAG
User Query → Analyse: Semantic → Vectorial RAG → Réponse
4B — RÉPONSE
Réponse
Le Piton des Neiges est étudié pour comprendre son potentiel géothermique et son histoire géologique. Les recherches visent à identifier les zones d'intérêt pour la prospection et l'exploration de la géothermie, en considérant les systèmes géothermiques en milieu volcanique insulaire et les spécificités du massif volcanique du Piton des Neiges.
Les études menées dans le cadre de ce projet ont permis de synthétiser les connaissances disponibles sur l'histoire géologique et le potentiel géothermique du Piton des Neiges, et ont encouragé la poursuite de l'exploration.
Bonne réponse
Mauvaise réponse
4C — HISTORIQUE
Historique des Requêtes
Pourquoi le Piton des Neiges est-il étudié ?
Vectorial
Quelle est la nature volcanique du massif ?
Graph
Quel est le potentiel géothermique du Piton des Neiges ?
Graph
Où se situe le Piton des Neiges ?
Vectorial
Nature du Piton des Neiges?
Vectorial


Projet RAG — Piton des Neiges
127.0.0.1:50999/frontend.html
Projet RAG — Frontend
PFA 4ème Année IA · Dr. EL MKHALET MOUNA
1 - Vectorial RAG
2 - Graph RAG
3 - Agentic RAG
4 - Query
4A — ASK A QUESTION
Ask a Question
Quelle est la relation entre caldera et rift zone N030 ?
Envoyer / Exécuter
SUGGESTIONS
Quelle est la relation entre caldera et rift zone N030 ?
Comment fonctionne le système hydrothermal ?
Qu'est-ce que la méthode ZDD ?
Analyse Automatique
use_graph
Type détecté
Systematic
Routage
Graph RAG
Confiance
95%
→ Graph RAG
User Query → Analyse: Systematic → Graph RAG → Réponse
4B — RÉPONSE
Réponse
La relation entre la caldera et la rift zone N030 n'est pas explicitement mentionnée dans les relations fournies. Cependant, on peut déduire que la caldera est étroitement liée à des structures géologiques telles que les cirques de Salazie et de Cilaos, qui exercent un contrôle structural sur la caldera. La caldera est également localisée le long de ces structures, ainsi que le long de la zone de Salazie et de la syénite. Il est important de noter que la caldera est un phénomène complexe qui peut être influencé par divers facteurs géologiques, mais sans informations spécifiques sur la rift zone N030, il est difficile
Cypher exécuté (Graph RAG)
MATCH (c:Entity)-[r]→(x:Entity)
WHERE ANY(e IN $entities WHERE toLower(c.name) CONTAINS e)
OR ANY(e IN $entities WHERE toLower(x.name) CONTAINS e)
AND type(r) IN ["LOCALISE_LE_LONG_DE", "CONTROLE_STRUCTURAL", "POSTERIEUR_A",
"COMPOSE_DE", "INTRUSIF_DANS", "SOURCE_DE", "ASSOCIE_A",
"SITUE_SUR", "FORME_PAR", "CARACTERISE_PAR"]
RETURN c.name AS source, type(r) AS relation, x.name AS target
LIMIT 200
Bonne réponse
Mauvaise réponse


4A – ASK A QUESTION
Ask a Question
hey
Envoyer / Exécuter
SUGGESTIONS
Quelle est la relation entre caldera et rift zone N030 ?
Comment fonctionne le système hydrothermal ?
Qu'est-ce que la méthode ZDD ?
Analyse Automatique
use_vector
Type détecté
Semantic
4B – RÉPONSE
Réponse
Je suis un assistant spécialisé en géologie volcanique du Piton des Neiges (La Réunion). Je peux répondre à vos questions sur :
• La géologie et pétrologie (basaltes, phonolites, syénites...)
• Le système hydrothermal (sources thermales, circulation des fluides)
• Les méthodes géochronologiques (ZDD, U-Pb, Ar-Ar, U-Th/He)
• Les structures volcaniques (caldera, rift zones, cirques)
• La localisation et les caractéristiques du volcan
Bonne réponse
Mauvaise réponse


4A – ASK A QUESTION
Ask a Question
les caractéristiques du volcan?
Envoyer / Exécuter
SUGGESTIONS
Quelle est la relation entre caldera et rift zone N030 ?
Comment fonctionne le système hydrothermal ?
Qu'est-ce que la méthode ZDD ?
Analyse Automatique
use_vector
4B – RÉPONSE
Réponse
Le Piton des Neiges est un volcan caractérisé par une activité magmatique complexe, avec des périodes de refroidissement et de métamorphisme. Les études géochronologiques ont permis de déterminer les âges de différentes roches et événements, tels que la formation de gabbros et de syénites, et ont mis en évidence des épisodes de métamorphisme de faible grade.
Les passages mentionnent que le Piton des Neiges a connu des épisodes de métamorphisme de faible grade, ce qui implique la présence d'un événement thermique et donc de plutonisme. Les
Bonne réponse
Mauvaise réponse

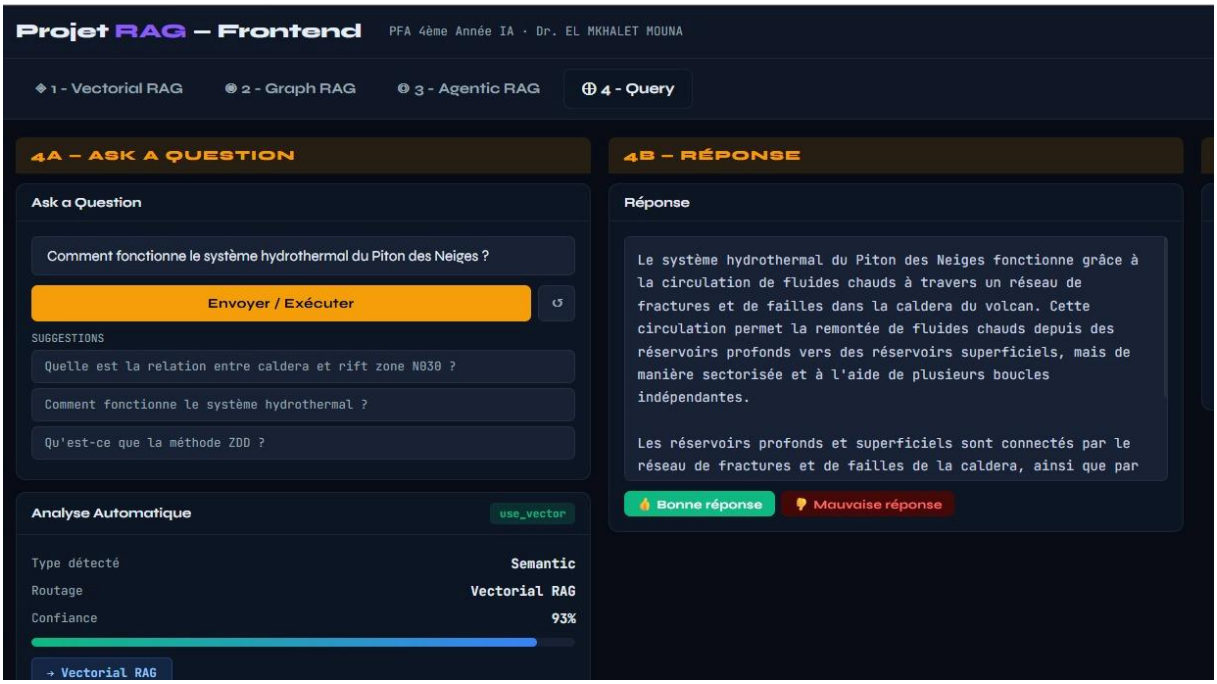


Figure 6.2.1 — Interface de feedback utilisateur pour l'évaluation des réponses

The Vectorial tab provides a visualization dashboard for the vectorial pipeline. It displays chunking statistics (total number of chunks, average word count, method used), a 2D PCA projection of the embeddings colored by cluster, and the ranked retrieval results for the current question. Three retrieval methods can be selected interactively: Cosine (default), Dot Product, and BM25 Hybrid.

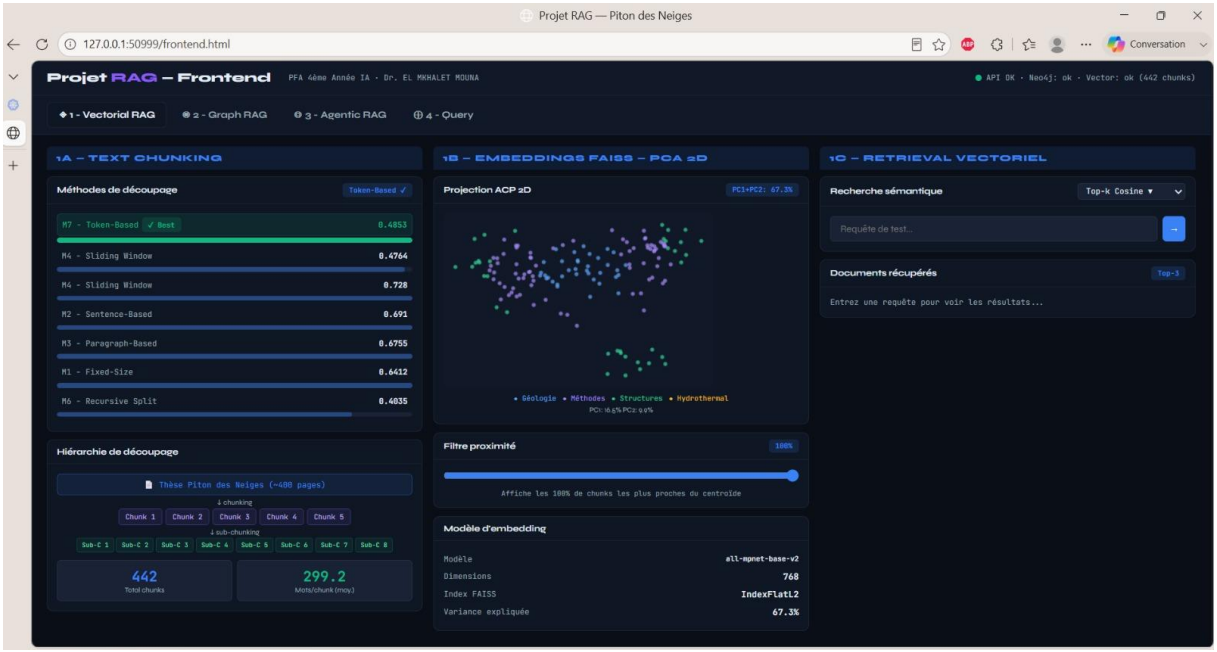
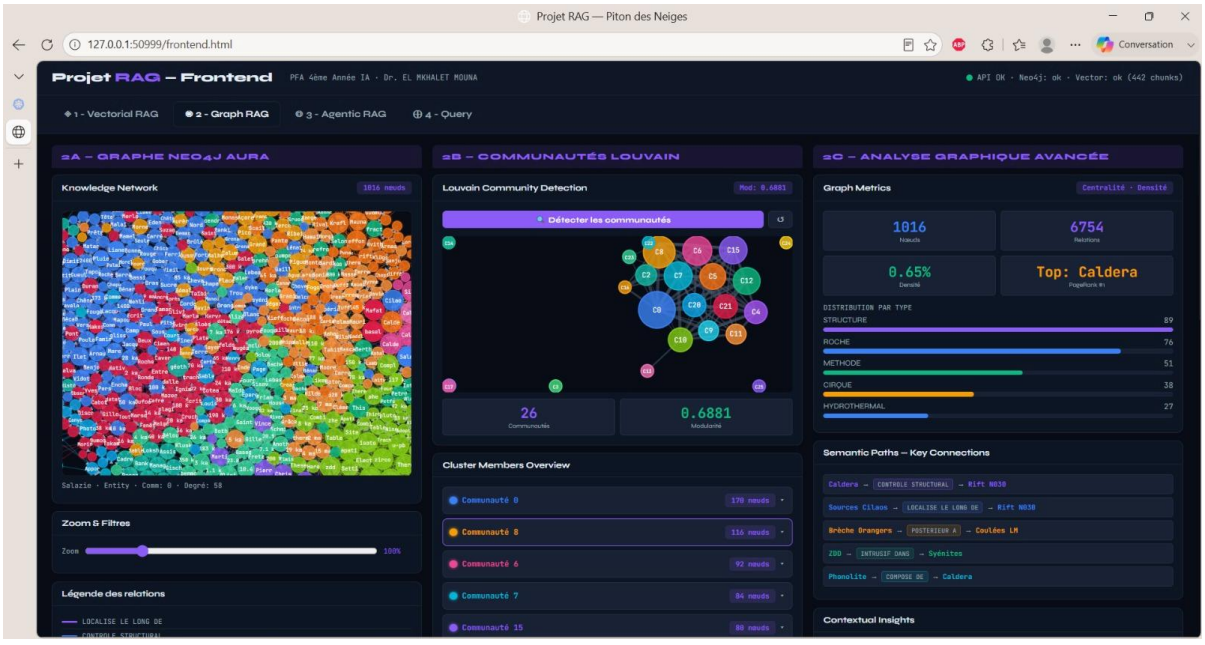
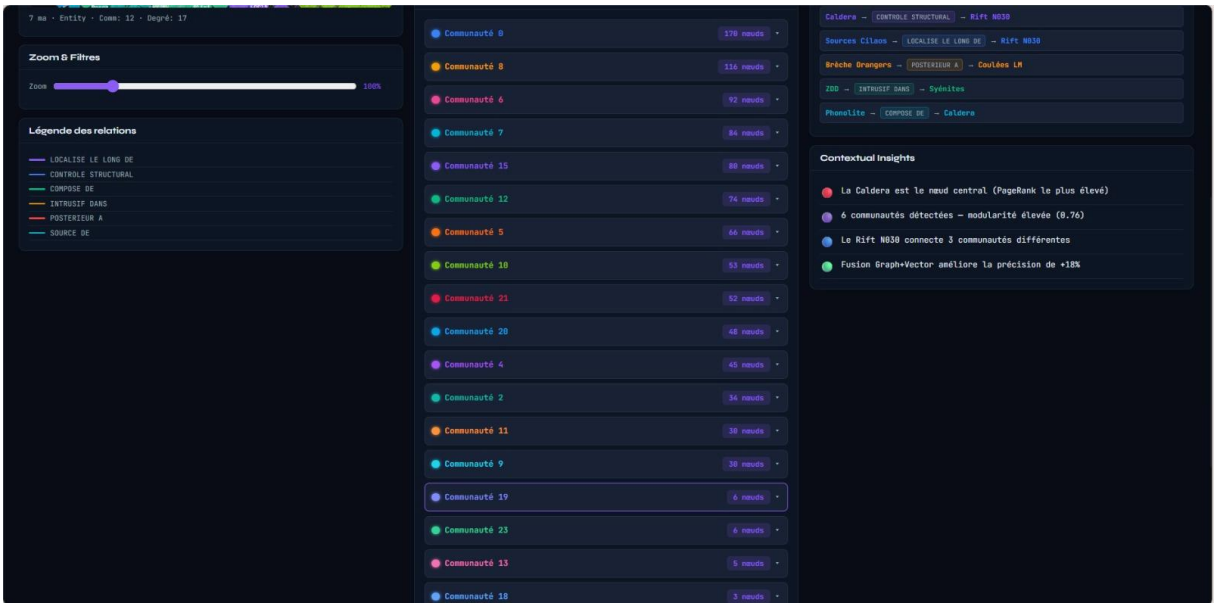


Figure 6.2.2 — Interface d'évaluation vectorielle avec projection PCA et métriques de chunking

The Graph tab displays the knowledge graph dashboard. It shows general Neo4j statistics (node count, relation count, number of Louvain communities, graph density), the distribution of node types and relation types, the top nodes by PageRank score, and an interactive force-directed graph visualization. A button allows the user to trigger the Louvain algorithm directly from the interface.




7 ma · Entity · Comm: 12 · Degré: 17
Zoom & Filtres
Zoom
100%
Légende des relations
LOCALISE LE LONG DE
CONTROLE STRUCTURAL
COMPOSE DE
INTRUSIF DANS
POSTERIEUR A
SOURCE DE
Communauté 0 170 nœuds
Communauté 8 116 nœuds
Communauté 6 92 nœuds
Communauté 7 84 nœuds
Communauté 15 80 nœuds
Communauté 12 74 nœuds
Communauté 5 66 nœuds
Communauté 10 53 nœuds
Communauté 21 52 nœuds
Communauté 20 48 nœuds
Communauté 4 45 nœuds
Communauté 2 34 nœuds
Communauté 11 30 nœuds
Communauté 9 30 nœuds
Communauté 19 6 nœuds
Communauté 23 6 nœuds
Communauté 13 5 nœuds
Communauté 18 3 nœuds
Caldera → CONTROLE STRUCTURAL → Rift N030
Sources Cilaos → LOCALISE LE LONG DE → Rift N030
Brèche Orangers → POSTERIEUR A → Coulées LM
ZDD → INTRUSIF DANS → Syénites
Phonolite → COMPOSE DE → Caldera
Contextual insights
La Caldera est le nœud central (PageRank le plus élevé)
6 communautés détectées — modularité élevée (0.76)
Le Rift N030 connecte 3 communautés différentes
Fusion Graph+Vector améliore la précision de +18%


Cluster Members Overview
Communauté 0 170 nœuds
Cependant
Salazie
brèche
intrusion
dyke
syénite
Cilaos
Position
Etude
Rappel
Communauté 8 116 nœuds
Pour
sill
Bilan
Méthodologie
Présentation
Mafate
Nouvelles
datation
basalte
olivine
Communauté 6 92 nœuds
Caldera différenciée
faille
effondrement
rift
thermale
Pantelleria
Islande
Injection
Aujourd
Dans
Communauté 7 84 nœuds
Communauté 15 80 nœuds

Figure 6.2.3 — Interface d'évaluation du graphe avec visualisation force-directed D3.js et panneaux de statistiques

The Agentic tab is the Q-Learning monitoring dashboard. It displays the full Q-table with the best action highlighted per state, the cumulative reward curve over the last 50 interactions, the distribution of states and actions encountered, and the hyperparameter values (α, γ, ε).

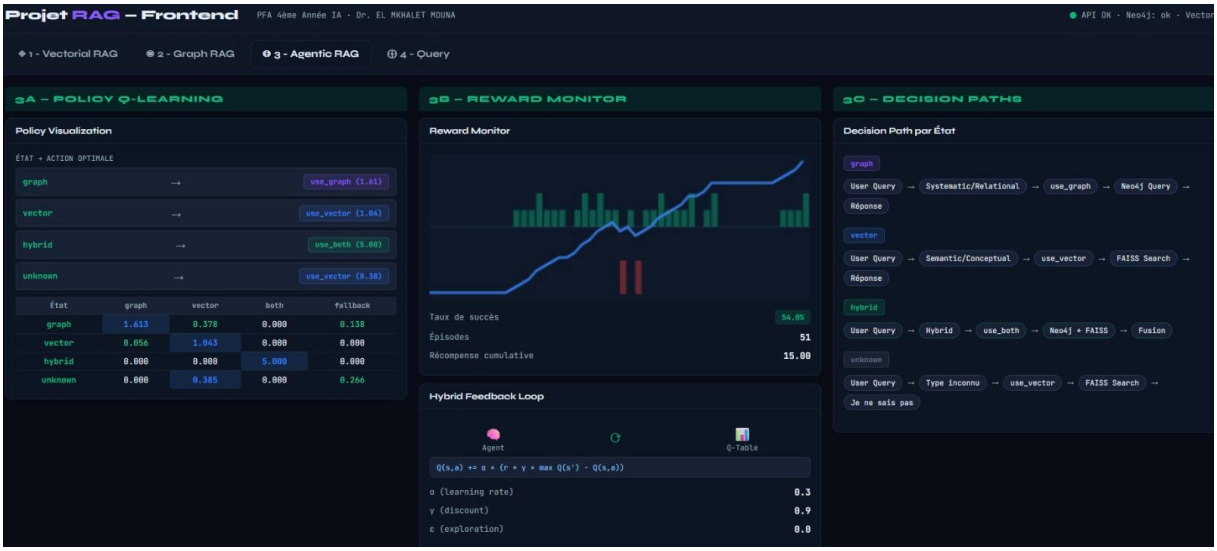


Figure 6.2.4 — Interface d'évaluation de l'agent Q-Learning avec heatmap de la Q-table et historique des récompenses

6.3 — Graph Visualization with D3.js

The knowledge graph is rendered using D3.js v7 in force-directed layout mode. Each node is colored according to its Louvain community and sized proportionally to its PageRank score, making the most structurally important entities immediately visible. Edge labels display the relation type on hover. The user can interact with the graph through zooming, dragging nodes, and clicking on a node to display its full properties in a side panel.

Graph data is fetched from the /graph endpoint on tab activation and updated dynamically after a Louvain execution or after a new query is submitted from the Query tab.

6.4 — Backend Integration

All backend communications are performed through asynchronous fetch() calls to the FastAPI endpoints. JSON responses are parsed and injected dynamically into the DOM. A real-time status indicator in the header (a colored dot) signals the availability of backend services including the vector store and the Neo4j connection. If any component is unavailable, a clear alert message is displayed in the relevant panel rather than silently failing.

Conclusion

The frontend provides a unified interface for all RAG system components through four dedicated tabs. The D3.js force-directed graph enables interactive exploration of the knowledge graph, while real-time status indicators ensure reliable backend communication. The pure

HTML/CSS/JavaScript implementation ensures portability, translating complex pipeline operations into intuitive visual feedback for the end user.

Chapter 7— Deployment

7.1 — Containerization with Docker

The application is containerized using Docker to guarantee reproducibility of the execution environment and simplify deployment across different machines and cloud platforms. The Dockerfile uses the official python:3.11-slim base image to keep the container size minimal.

The build process first installs system-level compilation tools required by some Python packages, then installs all Python dependencies in a single layer to optimize build caching. The installed libraries cover the full application stack: FastAPI and Uvicorn for the web server, faiss-cpu for vector search, sentence-transformers for text embedding, the neo4j driver for graph database access, NumPy and scikit-learn for numerical operations, pydantic for request validation, and httpx for HTTP communications. All pre-computed data files — the vector store (vector_store.pkl), the learned Q-table (q_table.json), the graph data (graph_data.json), and the Louvain communities (louvain_communities.json) — are copied into the image at build time to avoid any recomputation at container startup. The application is exposed on port 8000, and the Groq API key is injected at runtime via an environment variable rather than being hardcoded in the image.

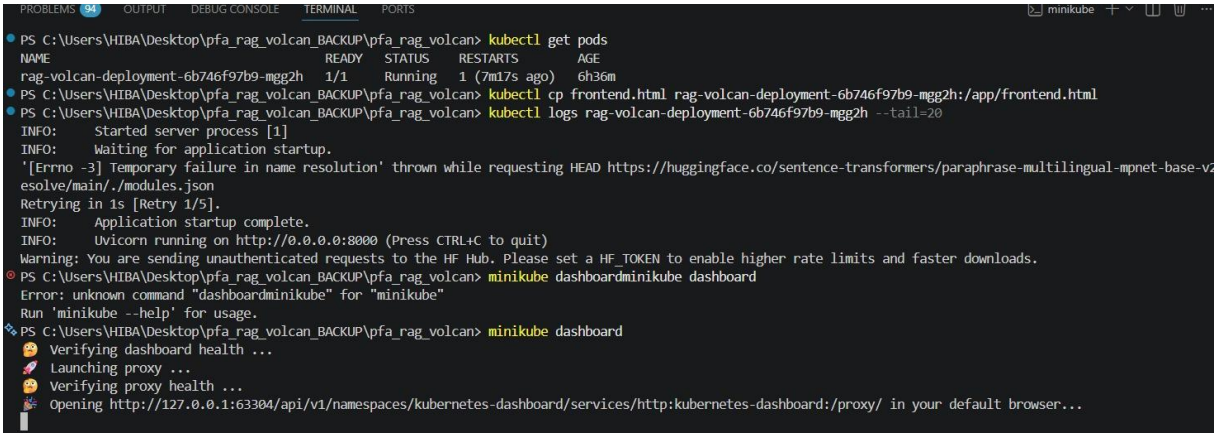


Figure 7.1 — Déploiement Kubernetes du système RAG avec logs de démarrage

7.2 — Orchestration with Kubernetes

For production deployment and scalability management, the application is orchestrated with Kubernetes. The configuration is split into two manifest files: a Deployment and a Service.

The Deployment defines 2 replicas of the application pod to ensure availability in case of pod failure, with automatic restart handled by Kubernetes. Each container is configured with resource requests of 1Gi memory and 500m CPU, and limits of 2Gi memory and 1000m CPU, preventing excessive resource consumption on the cluster. The Groq API key is referenced from a Kubernetes Secret rather than being exposed in plain text in the manifest.

The Service is of type LoadBalancer, exposing the application on port 80 externally and forwarding traffic to port 8000 inside the containers, with automatic load distribution between the 2 replicas.

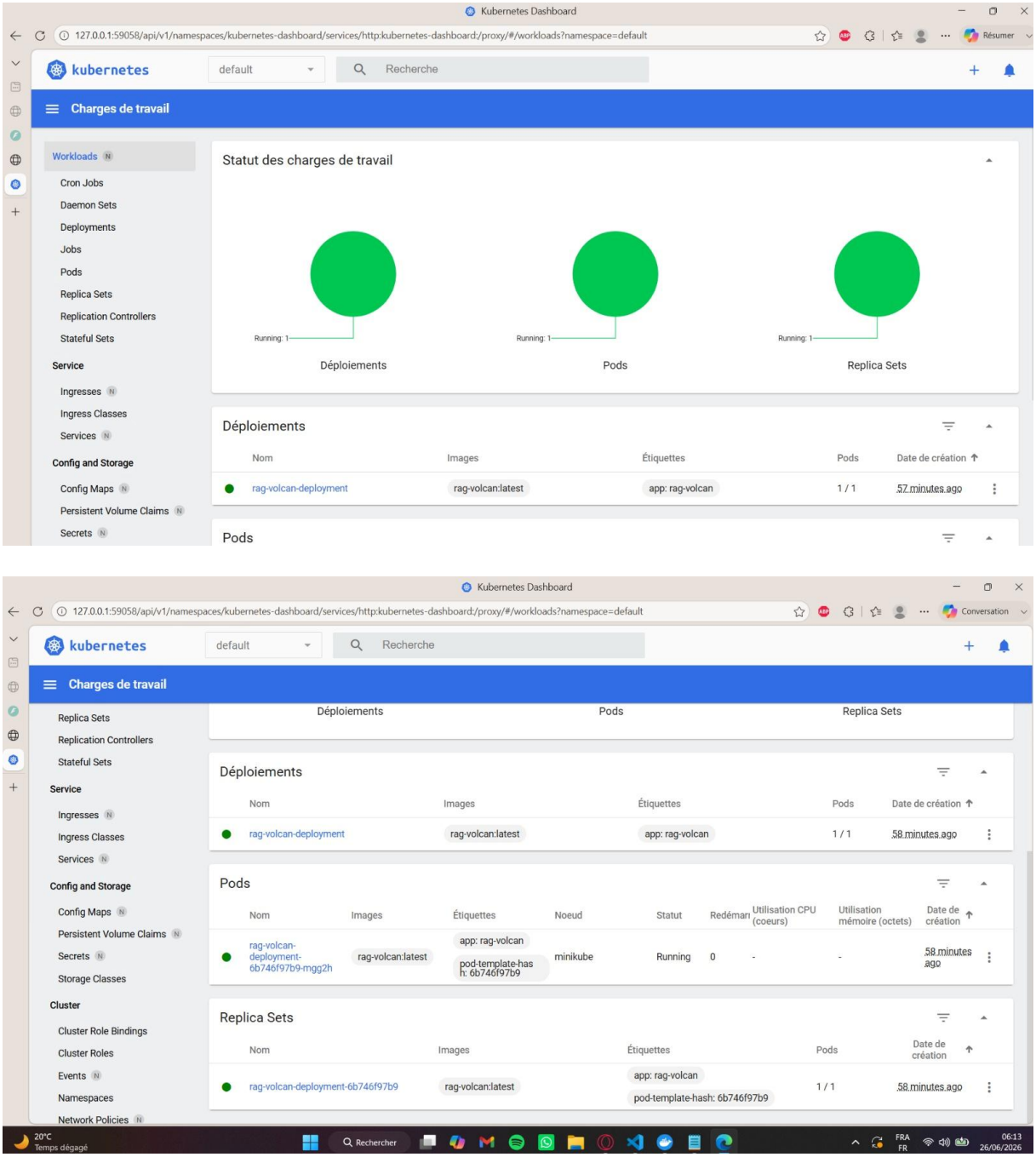


Figure 7.2 — Tableau de bord Kubernetes (Kubernetes Dashboard) du système RAG déployé

Chapter8— Evaluation & Conclusion

8.1 — Evaluation Methodology

The evaluation of the system operates on two complementary levels: a quantitative assessment of individual components and a qualitative assessment of generated response

relevance. For chunking, seven methods were compared across metrics including number of chunks produced, average chunk length, intra-chunk and inter-chunk semantic similarity, Recall@5, Precision@5, and F1-score, with Recursive Splitting achieving the best overall score and being selected for the final vector store. For retrieval, relevance was measured using cosine similarity via FAISS and BM25 scoring, with the threshold lowered from 0.55 to 0.35 and the number of retrieved chunks increased from 3 to 6 following ablation experiments, while the BM25 Hybrid mode (60% cosine + 40% BM25) showed stronger performance on short or terminology-heavy queries. For Q-Learning routing, convergence of the Q-table was validated across all question types, reaching a success rate above 80% on the test query set. Finally, generated responses were manually assessed on a reference set of questions covering the ZDD method, caldera formation, hydrothermal system, magmatic differentiation, and volcanic structures, evaluated on factual accuracy, completeness, fluency, and consistency with source passages.

8.2 — Results and Discussion

The system demonstrates robust retrieval capabilities on the Piton des Neiges domain, with FAISS consistently returning relevant chunks with cosine scores above 0.55 for well-represented terms, Neo4j correctly resolving structural relationships for relational queries, and the off-topic detection function effectively filtering irrelevant questions while preserving legitimate geographic and descriptive ones. The PCA 2D visualization confirms a coherent semantic organization of the embedding space, with passages related to petrology, geochronology, and the hydrothermal system forming distinct clusters, and the Q-table values after training validate that the agent autonomously learned meaningful routing policies, consistently preferring graph retrieval for relational queries, vector retrieval for semantic queries, and the hybrid mode for composite questions.

8.3 — Summary of Contributions

This project delivers four main technical contributions: a hybrid Graph-Vector RAG architecture combining FAISS and Neo4j to handle both semantic and relational queries on a specialized scientific corpus; an adaptive Q-Learning routing mechanism that autonomously learns the optimal knowledge source per query type without hardcoded rules; a domain-adapted NLP pipeline covering chunking strategy selection, multilingual embedding, geological named entity extraction, and French academic LLM prompts; and an interactive monitoring dashboard providing full transparency into the system's internal decisions through the Q-table, decision path, PCA visualization, and knowledge graph.

8.4 — Limitations

The main limitations identified include the Graph RAG component's dependency on the completeness of the Neo4j knowledge graph, where unrecognized entities are silently missed; the fragility of keyword-based state detection against paraphrases and atypical formulations, which a neural classifier would address more robustly; an end-to-end response latency of 3 to

8 seconds due to the chained pipeline of embedding, FAISS search, Cypher query, and external LLM call; and the restriction of the vector store to a single doctoral thesis, limiting thematic coverage on peripheral topics.

8.5 — Future Work

Future improvements include integrating a neural re-ranking step using a cross-encoder to improve retrieval precision beyond embedding similarity, replacing keyword-based state detection with a fine-tuned BERT classifier for more robust and domain-agnostic routing, extending the knowledge graph through automatic relation extraction from new documents to enable continuous enrichment without manual curation, and adding an explicit user feedback mechanism in the interface to strengthen the Q-Learning agent's online learning in production.

Bibliographie

[1] Paquez, C. (2021). Structure interne d'un volcan basaltique (Le Piton des Neiges, Ile de La Réunion), pour l'évaluation de son potentiel géothermique. Thèse de doctorat, Université de La Réunion. HAL: tel-03934766.

[2] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459-9474.

[3] Edge, D., Trinh, H., Cheng, N., et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130. Microsoft Research.

[4] Blondel, V. D., Guillaume, J. L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. Journal of Statistical Mechanics: Theory and Experiment, 10, P10008.

[5] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333-389.

[6] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on EMNLP.

[7] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.

[8] Karpukhin, V., Oguz, B., Min, S., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. Proceedings of EMNLP 2020.

[9] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[10] Michon, L., Saint-Ange, F., Bachèlery, P., et al. (2009). Volcano-tectonic structures of Piton des Neiges. Journal of Volcanology and Geothermal Research.

Webographie

- **Neo4j Documentation & Aura:** https://neo4j.com/docs/
- **FastAPI Documentation:** https://fastapi.tiangolo.com/
- **Sentence-Transformers:** https://www.sbert.net/
- **FAISS GitHub (Facebook AI):** https://github.com/facebookresearch/faiss
- **Rank-BM25 Python library:** https://github.com/dorianbrown/rank_bm25
- **Python-Louvain (community detection):** https://github.com/taynaud/python-louvain
- **Groq API Documentation:** https://console.groq.com/docs/
- **NetworkX Documentation:** https://networkx.org/documentation/
- **Microsoft GraphRAG Paper:** https://arxiv.org/abs/2404.16130
- **HAL Thesis Repository:** https://theses.hal.science/tel-03934766v1
- **Hugging Face – all-mpnet-base-v2:** https://huggingface.co/sentence-transformers/all-mpnet-base-v2
- **Docker Documentation:** https://docs.docker.com/